

MINERÍA DE DATOS

Maximiliano Ojeda

muojeda@uc.cl



IIC-2433

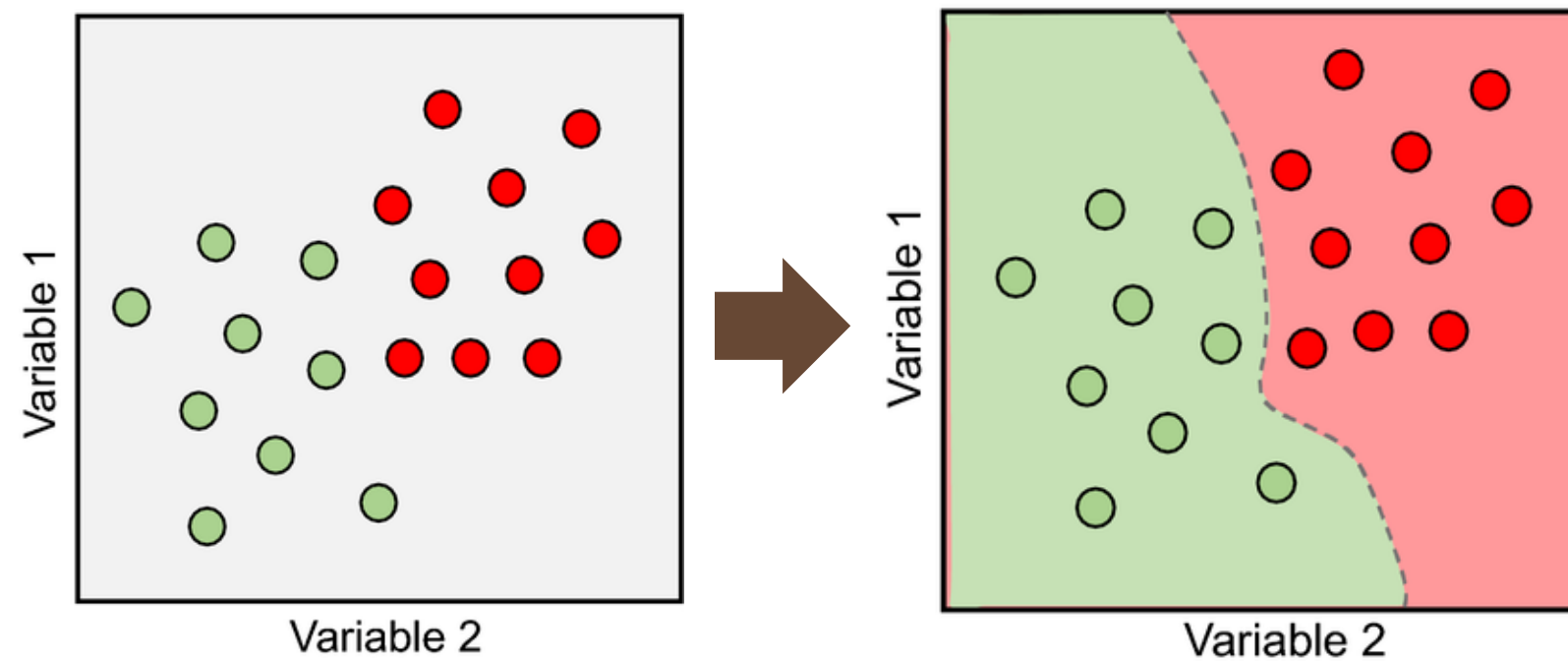


Aprendizaje No Supervisado

Tipos de Aprendizaje

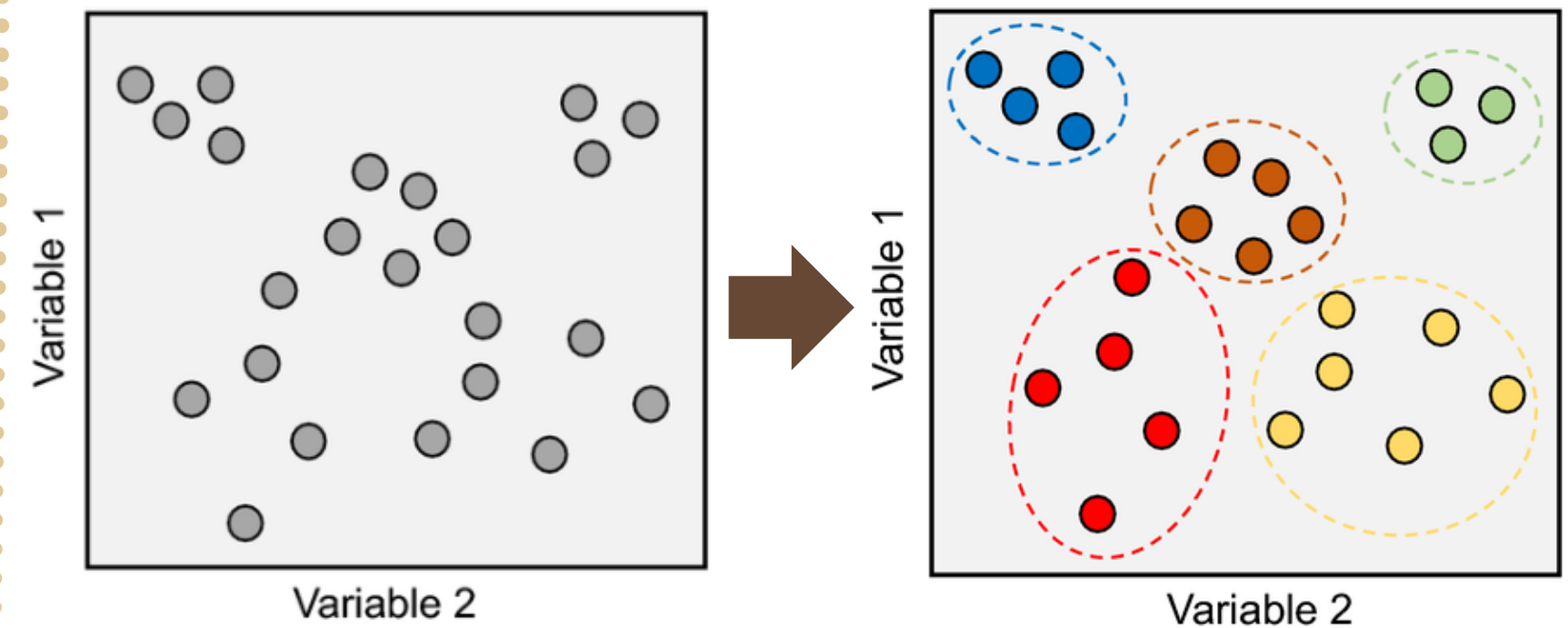
Aprendizaje Supervisado

Los datos están etiquetados (existen un X y un Y).
Los modelos aprenden a predecir la etiqueta Y



Aprendizaje No Supervisado

Solamente contamos con X, Los modelos buscan patrones ocultos o estructuras



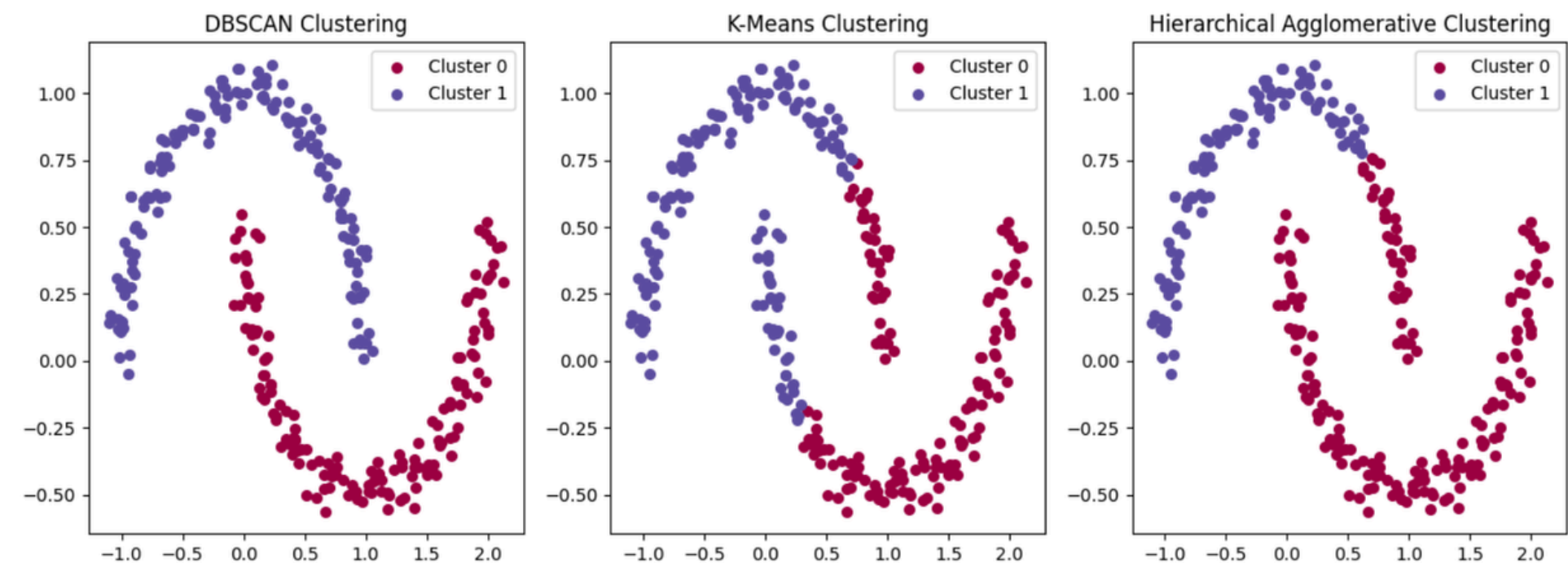
Clustering

Clustering es una técnica de aprendizaje no supervisado cuyo objetivo es agrupar un conjunto de datos en grupos (clusters) de manera que:

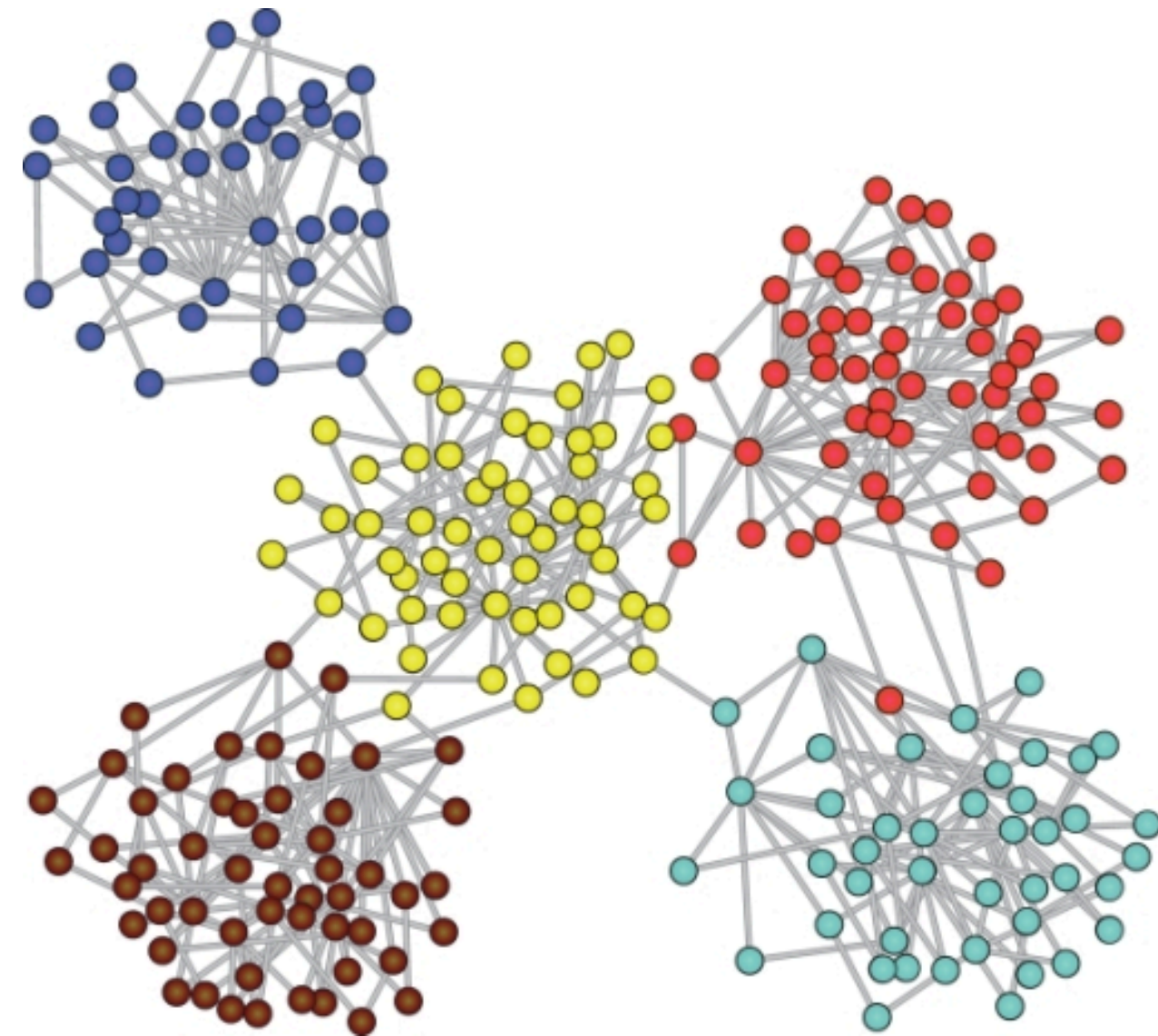
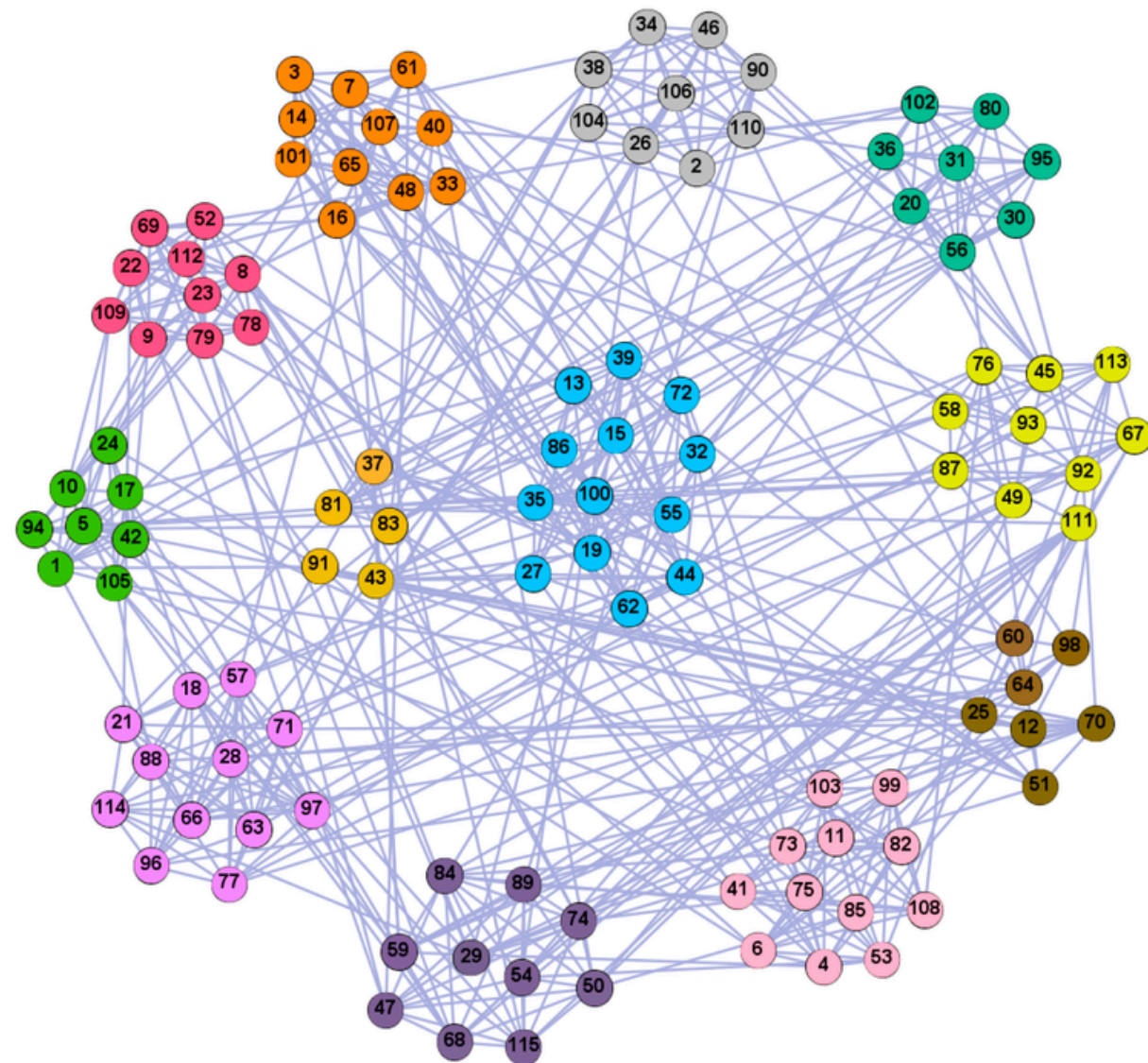
- Los elementos dentro de un mismo grupo sean lo más similares entre sí.
- Los elementos de grupos diferentes sean lo más distintos posible.

Aplicaciones

- Segmentación de clientes
- Análisis de imágenes
- Detección de anomalías
- Agrupación de documentos/textos.
- Biología y medicina.



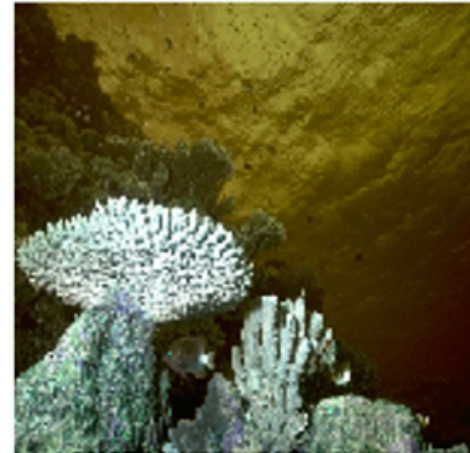
Clustering



Clustering



Segmentation



Segmentation



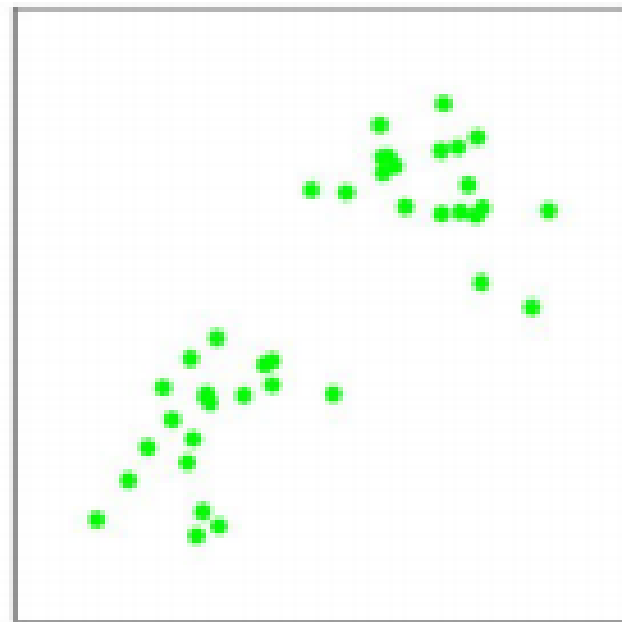
Segmentation



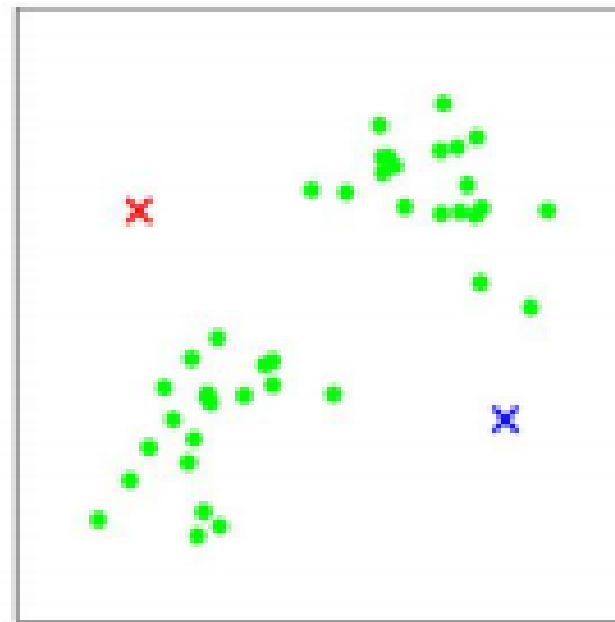


K-Means

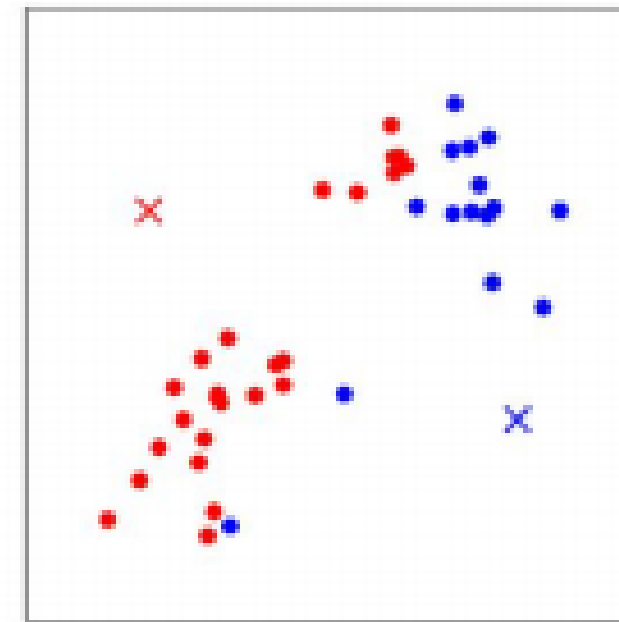
K-Means



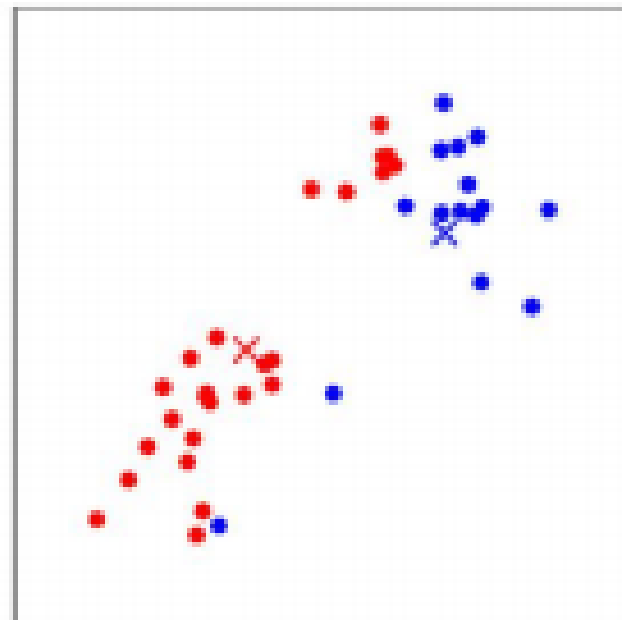
(a)



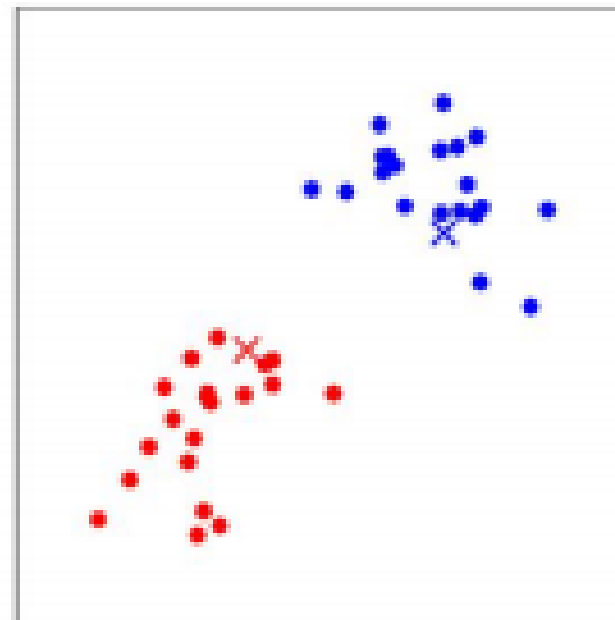
(b)



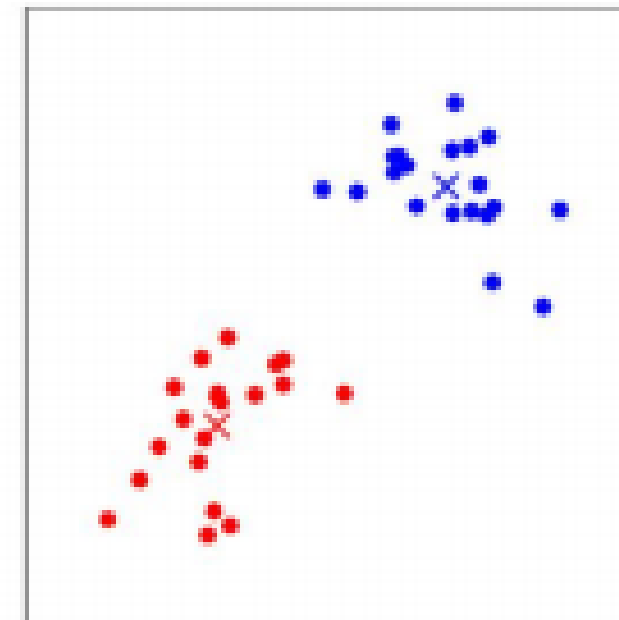
(c)



(d)



(e)

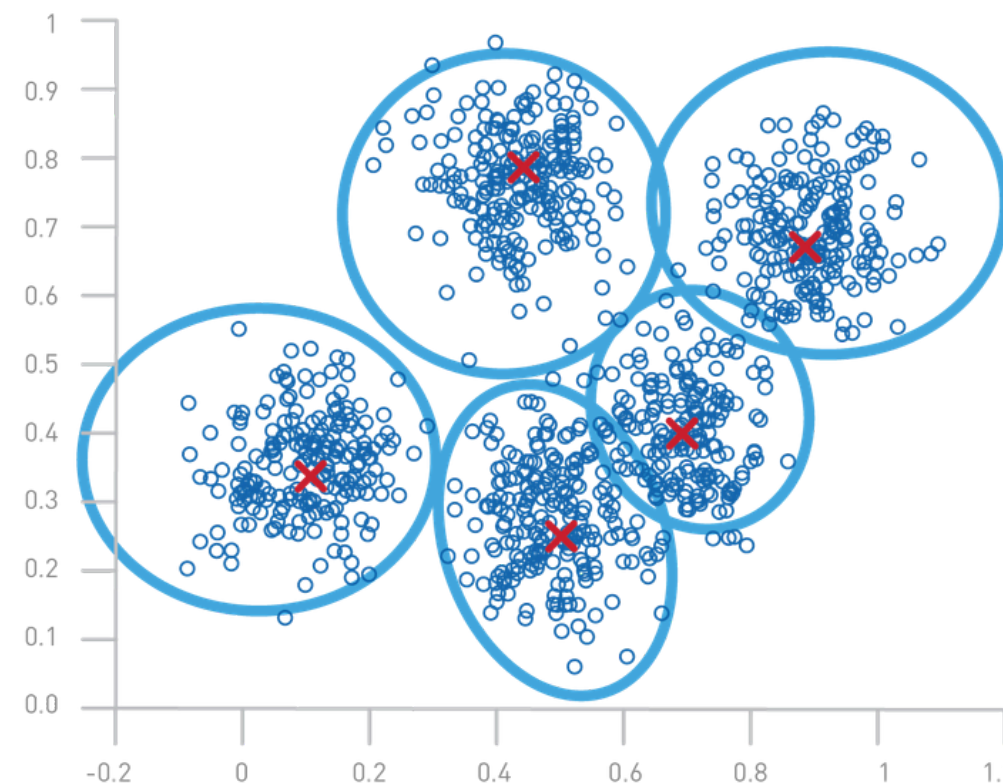


(f)

K-Means

K-Means funciona como un algoritmo iterativo que divide los datos en k grupos (clusters) según su cercanía:

1. **Elegir k :** decidir cuántos clusters va a encontrar.
2. **Inicializar centroides:** se colocan k puntos al azar (los “centros” de cada cluster).
3. **Asignar puntos:** cada dato se asigna al **centroide más cercano** (ejemplo, distancia euclidiana).
4. **Actualizar centroides:** calcular el promedio de los puntos en cada cluster y se mueve el centroide a esa posición.
5. **Repetir** pasos 3–4 hasta que los centroides ya no cambien (o número máximo de iteraciones).



K-Means

Problema

Tenemos el conjunto de datos:

$$X = \{x_1, x_2, \dots, x_n\}, \quad x_i \in \mathbb{R}^d$$

El objetivo es dividirlos en **k** clusters

$$C = \{C_1, C_2, \dots, C_k\}.$$

Función Objetivo

Minimizar las distancias entre los puntos y el centroide de su cluster:

$$J = \sum_{j=1}^k \sum_{x_i \in C_j} \|x_i - \mu_j\|^2$$

Algoritmo iterativo

Inicialización: elige aleatoriamente **k** centroides $\mu_1, \dots, \mu_k$

Asignación: cada punto se asigna al cluster más cercano:

$$C_j = \{x_i : \|x_i - \mu_j\|^2 \leq \|x_i - \mu_l\|^2 \quad \forall l\}$$

Actualización: recalcular el centroide de cada cluster:

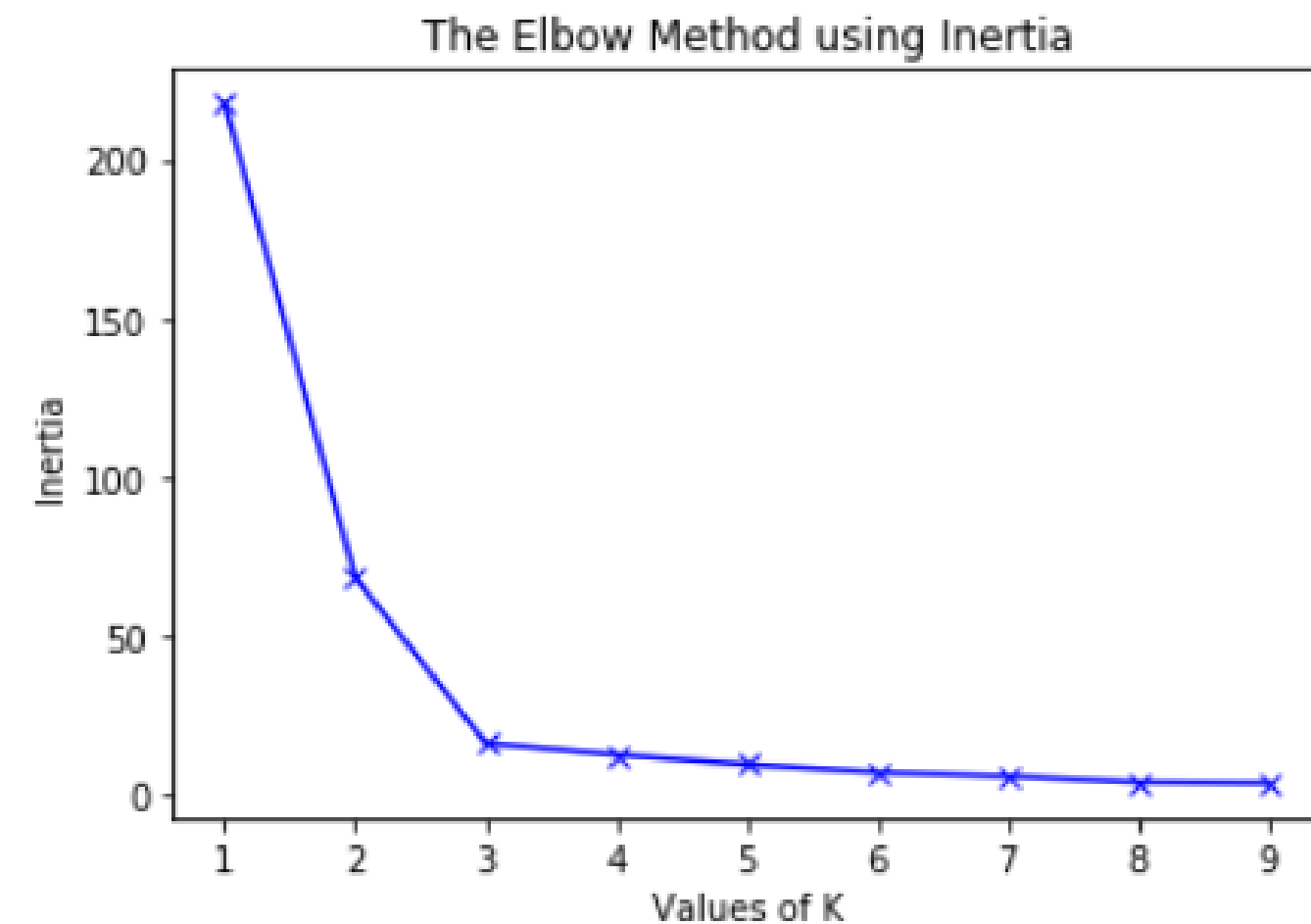
$$\mu_j = \frac{1}{|C_j|} \sum_{x_i \in C_j} x_i$$

Repetir hasta que los centroides no cambien.

Método del codo (Elbow)

- Calcular la función objetivo (también llamada inercia) para distintos valores de **k**.
- A medida que aumenta **k**, la inercia disminuye (más clusters = menor error).
- Se elige el **k** en el que la mejora empieza a ser marginal → el “codo” de la curva.

$$\text{Inertia}(k) = \sum_{j=1}^k \sum_{x_i \in C_j} \|x_i - \mu_j\|^2$$



Silhouette Score

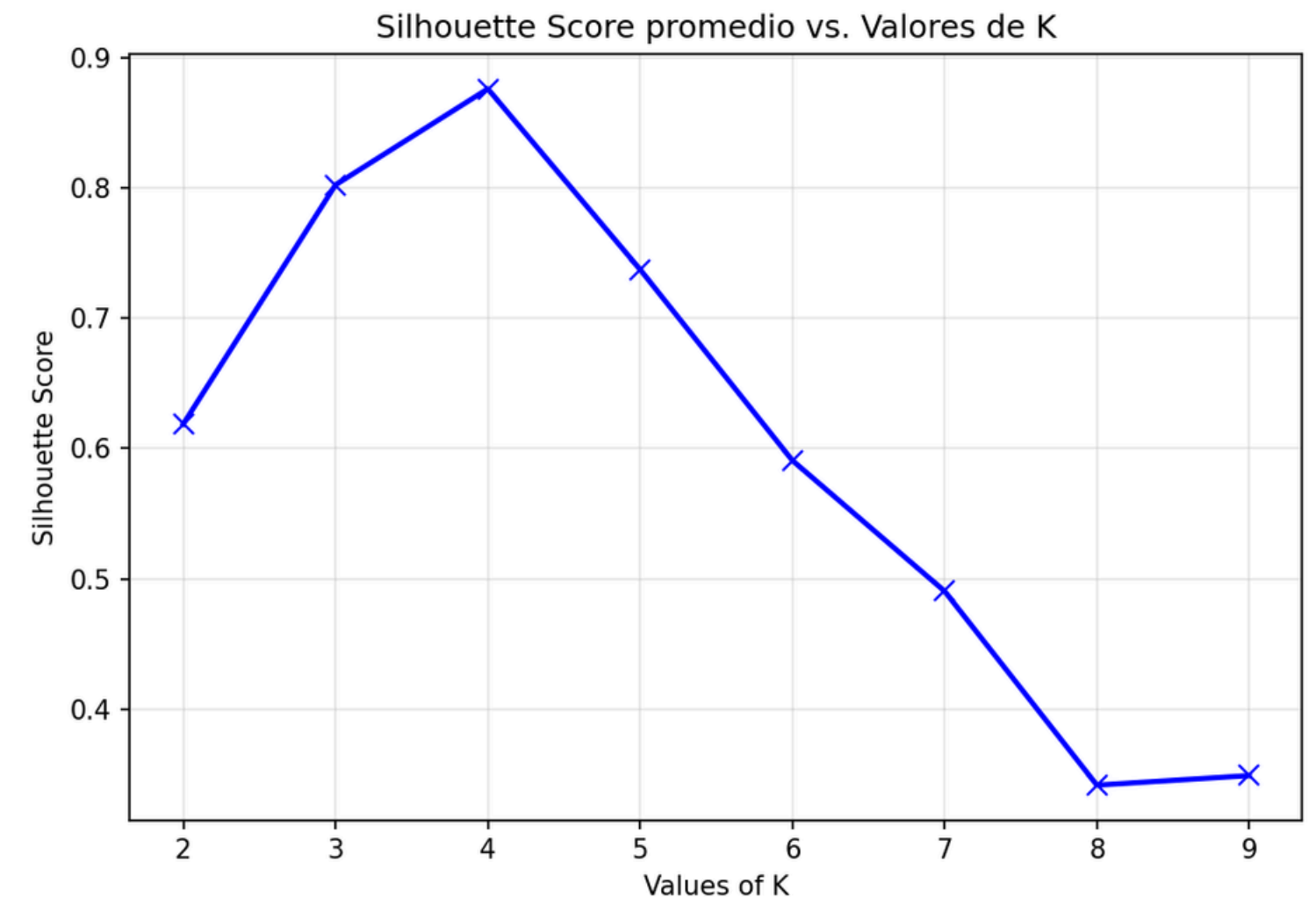
El **Silhouette Score** mide qué tan bien queda cada punto asignado a su cluster, comparando:

- $a(i)$: distancia promedio del punto a los demás puntos de su mismo cluster (**cohesión**)
- $b(i)$: distancia promedio del punto a los puntos del cluster vecino más cercano (**separación**)

$$s(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))}$$

$$\text{Silhouette}(k) = \frac{1}{n} \sum_{i=1}^n s(i)$$

A diferencia del método del codo, el Silhouette Score permite comparar directamente la calidad de distintos valores de k : se elige el k que maximiza el score promedio.



K-Means

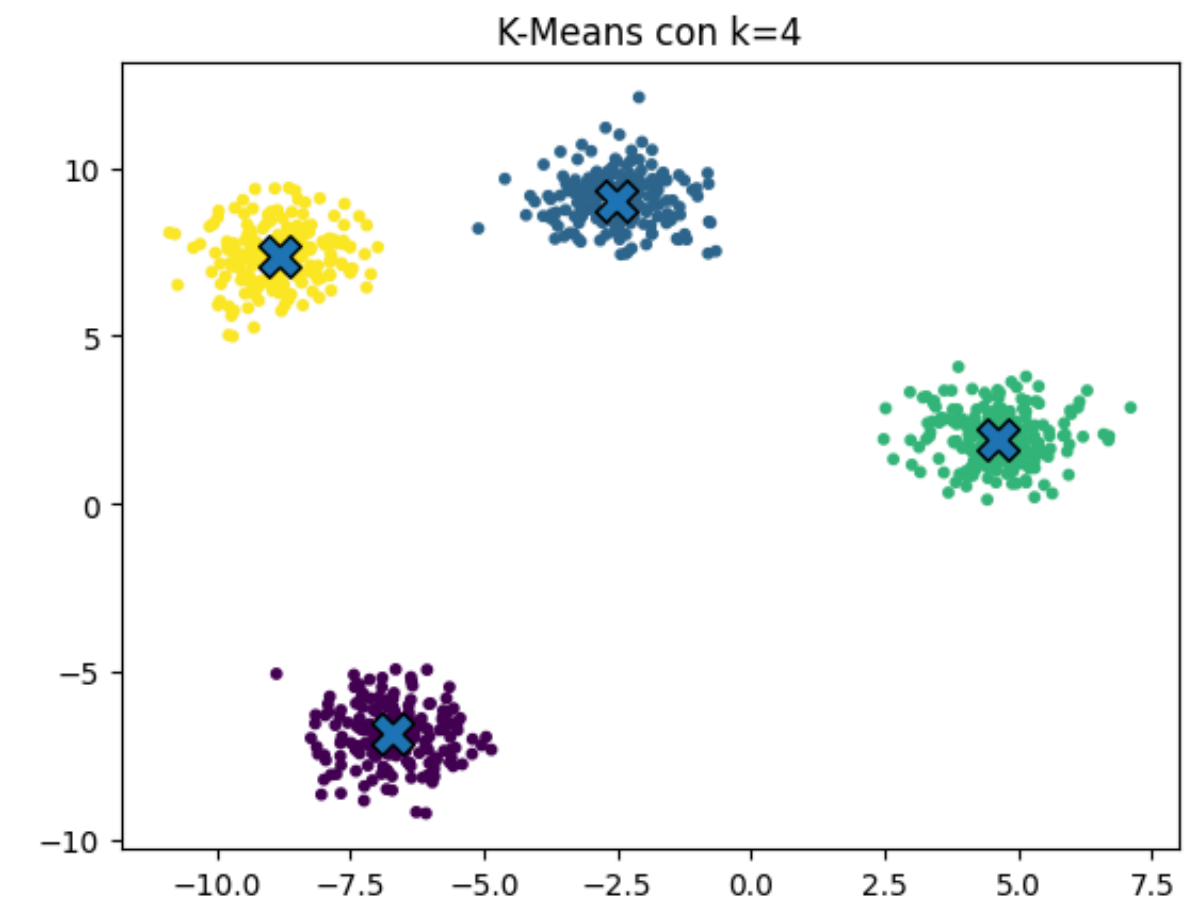
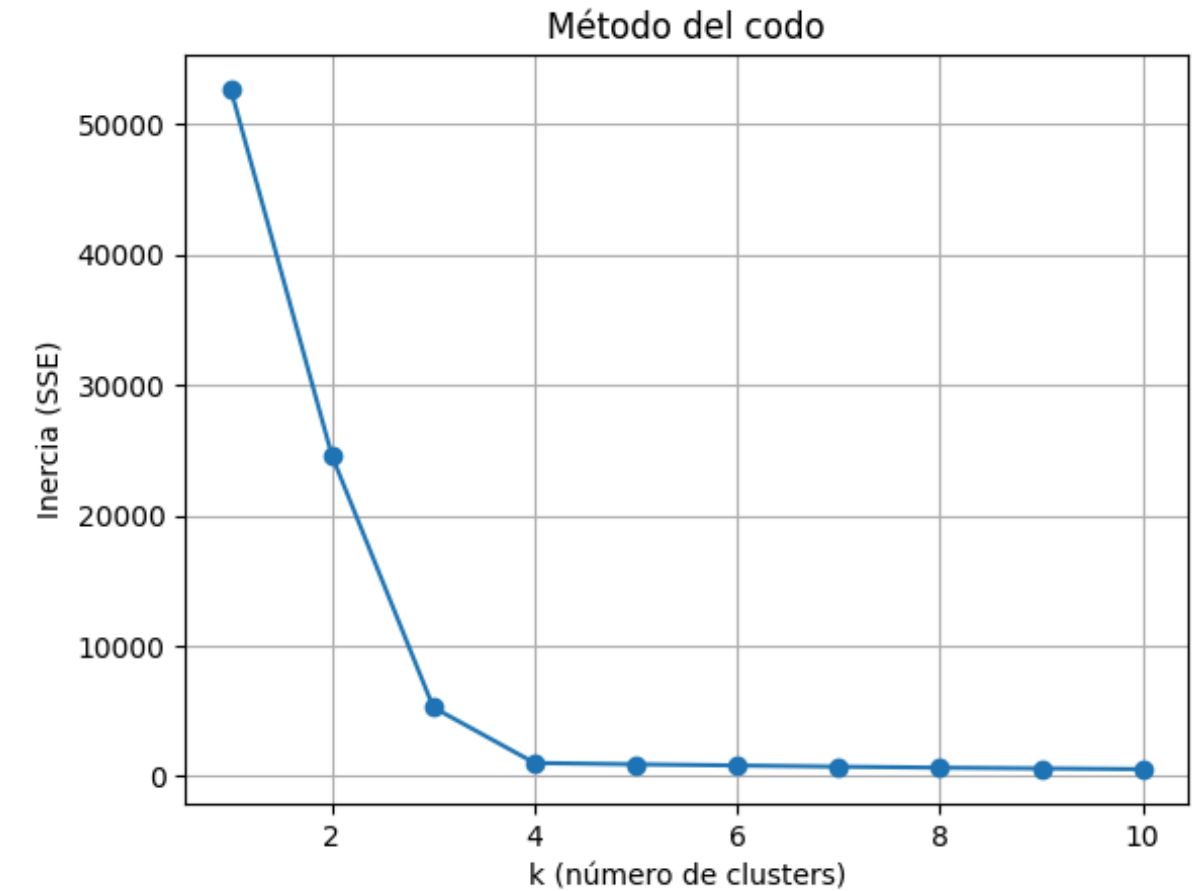
```
from sklearn.cluster import KMeans

X, y_true = make_blobs(n_samples=800, centers=4, cluster_std=0.80)

# 2) Cálculo de inercia para distintos k
Ks = range(1, 11)
inertias = []
for k in Ks:
    km = KMeans(n_clusters=k, n_init=10, random_state=42)
    km.fit(X)
    inertias.append(km.inertia_)

# Eligir k mirando el "codo" de la curva
k_opt = 4

# K-Means con k_opt
km_final = KMeans(n_clusters=k_opt, n_init=10, random_state=42)
labels = km_final.fit_predict(X)
centers = km_final.cluster_centers_
print("Inercia con k_opt:", km_final.inertia_)
print("Centroides:\n", centers)
```





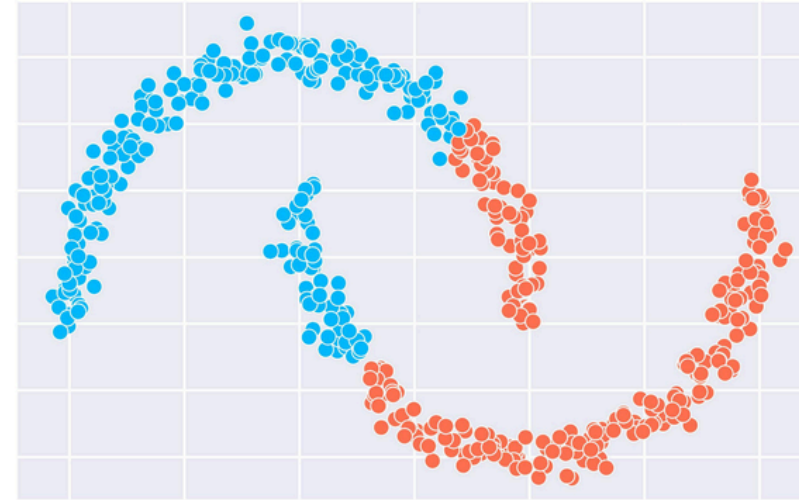
DBSCAN

¿Por qué necesitamos otro algoritmo?

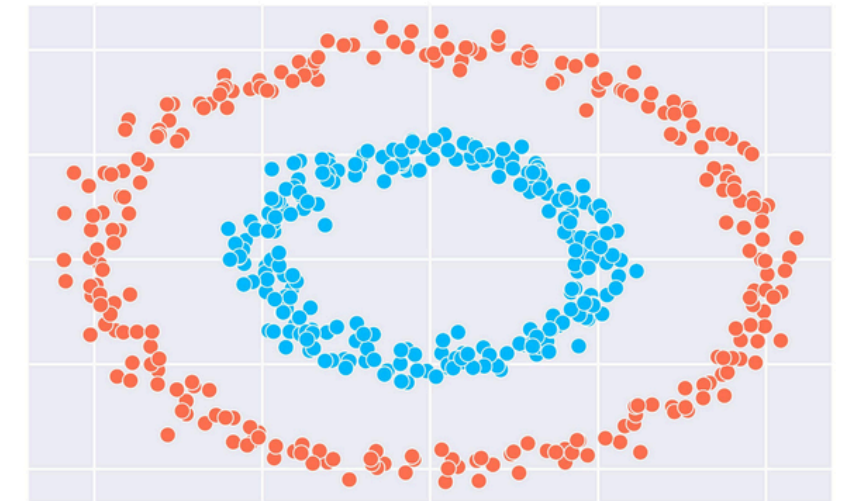
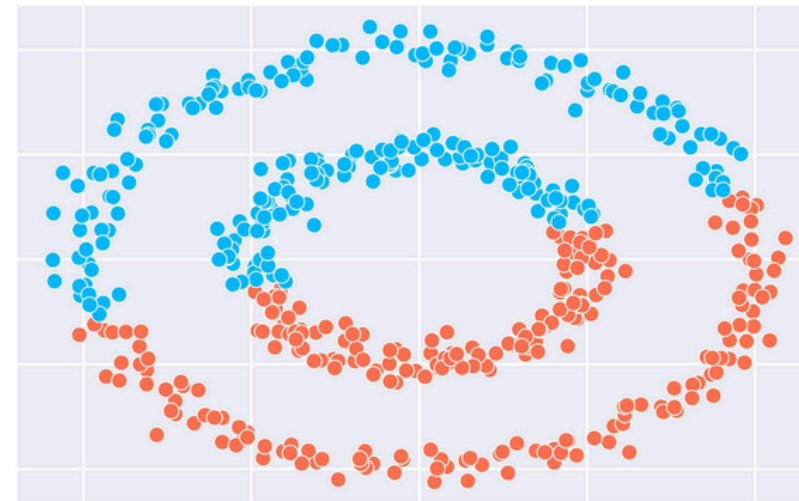
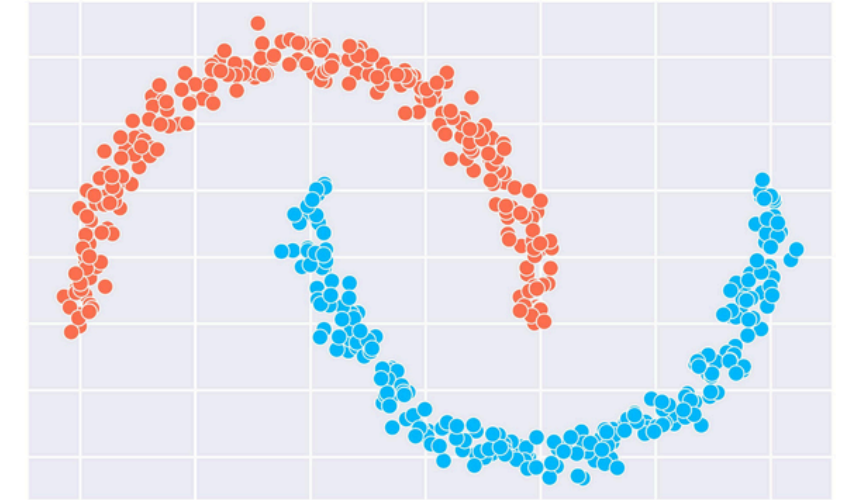
Limitaciones de K-Means

- Asume que los clusters son aproximadamente esféricos y de tamaño similar.
- Es **sensible a outliers**, un solo dato muy alejado puede desplazar fuertemente un centroide.

KMeans



DBSCAN



DBSCAN

DBSCAN (Density-Based Spatial Clustering of Applications with Noise) funciona de manera distinta a K-Means: en lugar de depender de centroides y un número fijo de clusters, agrupa puntos según su densidad en el espacio.

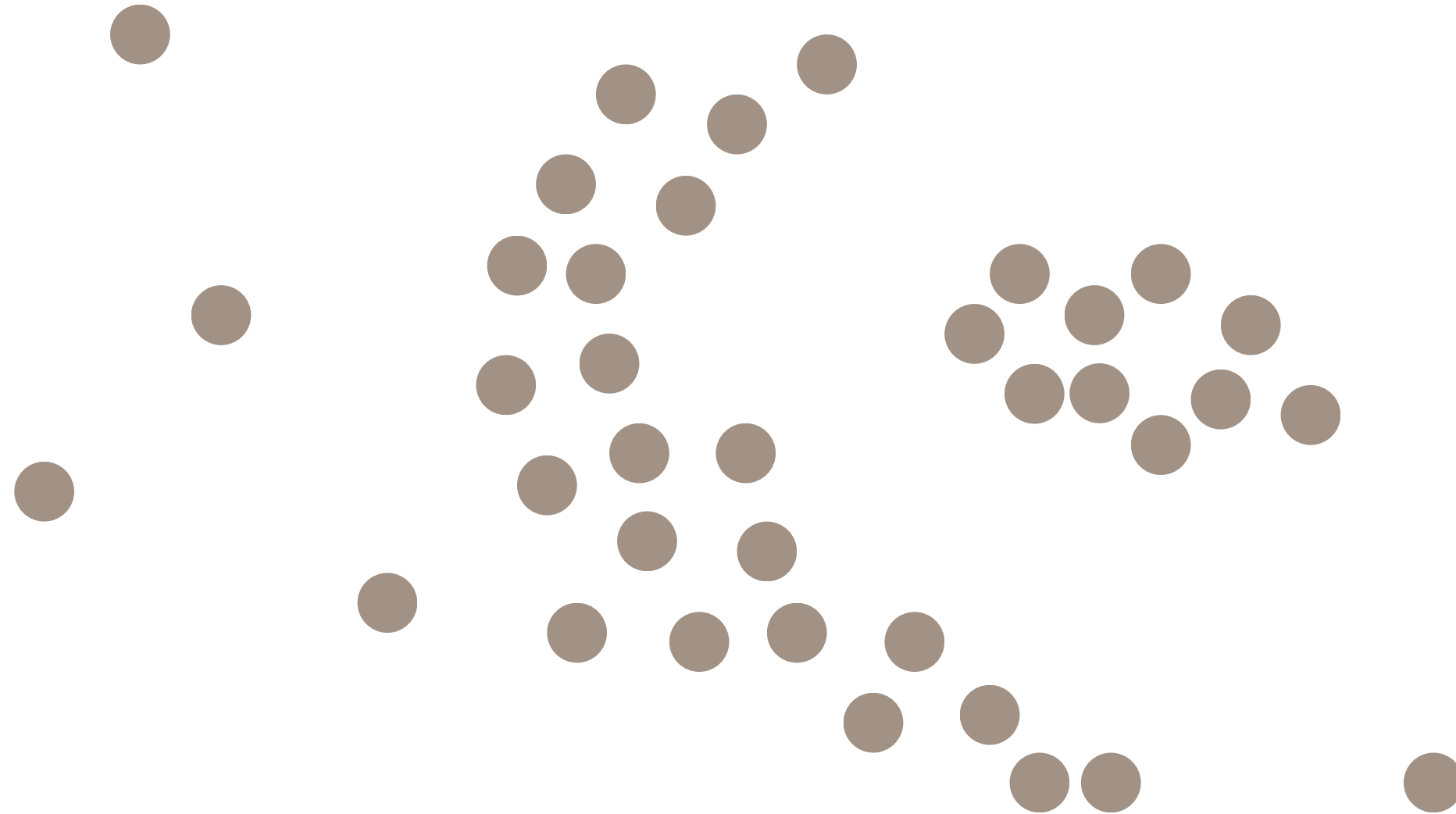
Parámetros clave

- **ϵ (eps):** radio de vecindad.
- **minPts:** número mínimo de puntos necesarios en un radio ϵ para ser considerado.

Con esto, **los puntos se clasifican** como:

1. **Núcleo (core point):** tiene al menos **minPts** vecinos dentro del radio ϵ .
2. **Borde (border point):** está dentro de la vecindad ϵ de un punto núcleo, pero no cumple por sí mismo con minPts.
3. **Outlier:** no es núcleo ni borde (queda fuera de clusters).

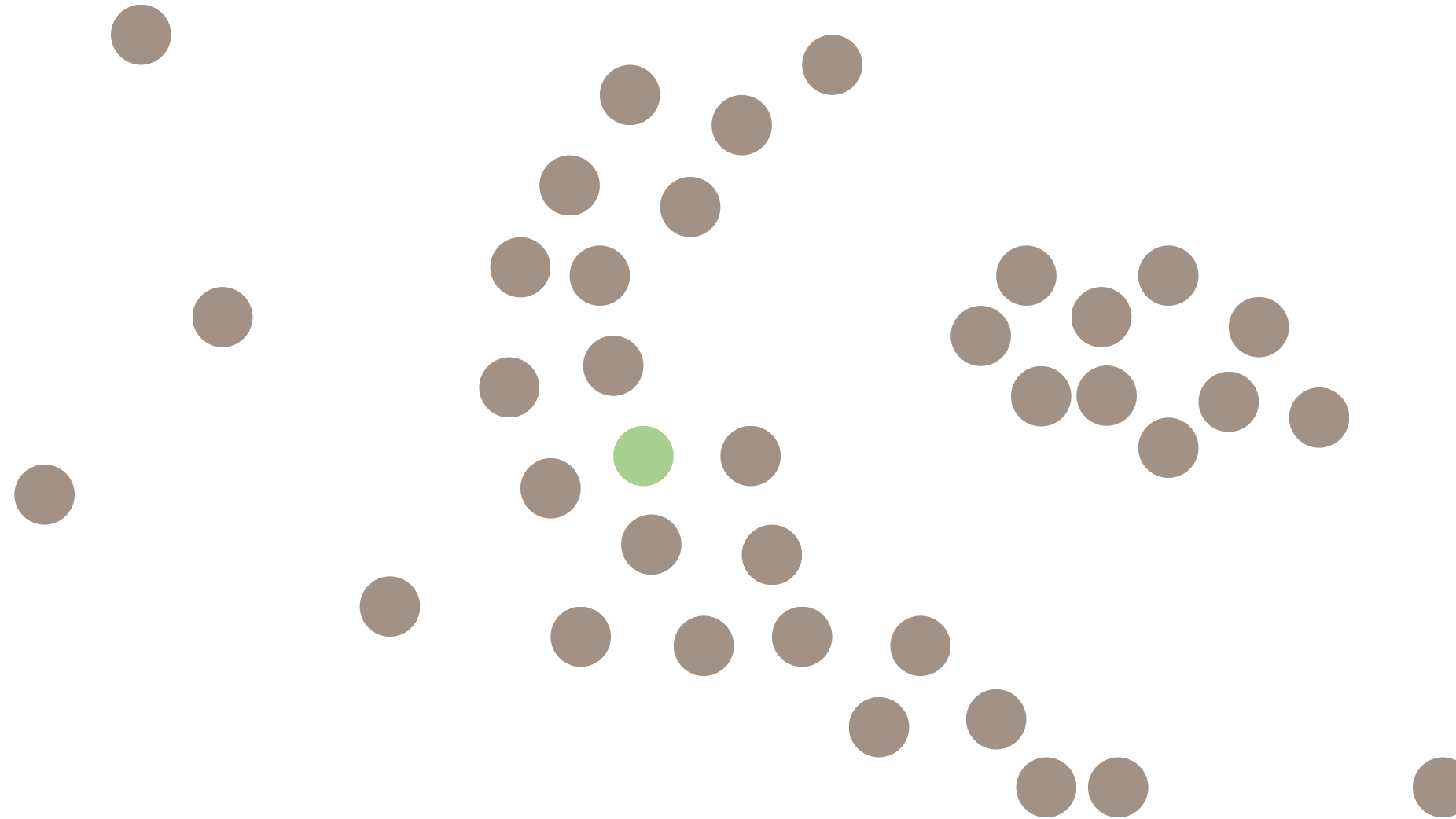
DBSCAN



Vamos a clusterizar este conjunto de puntos con DBSCAN

DBSCAN

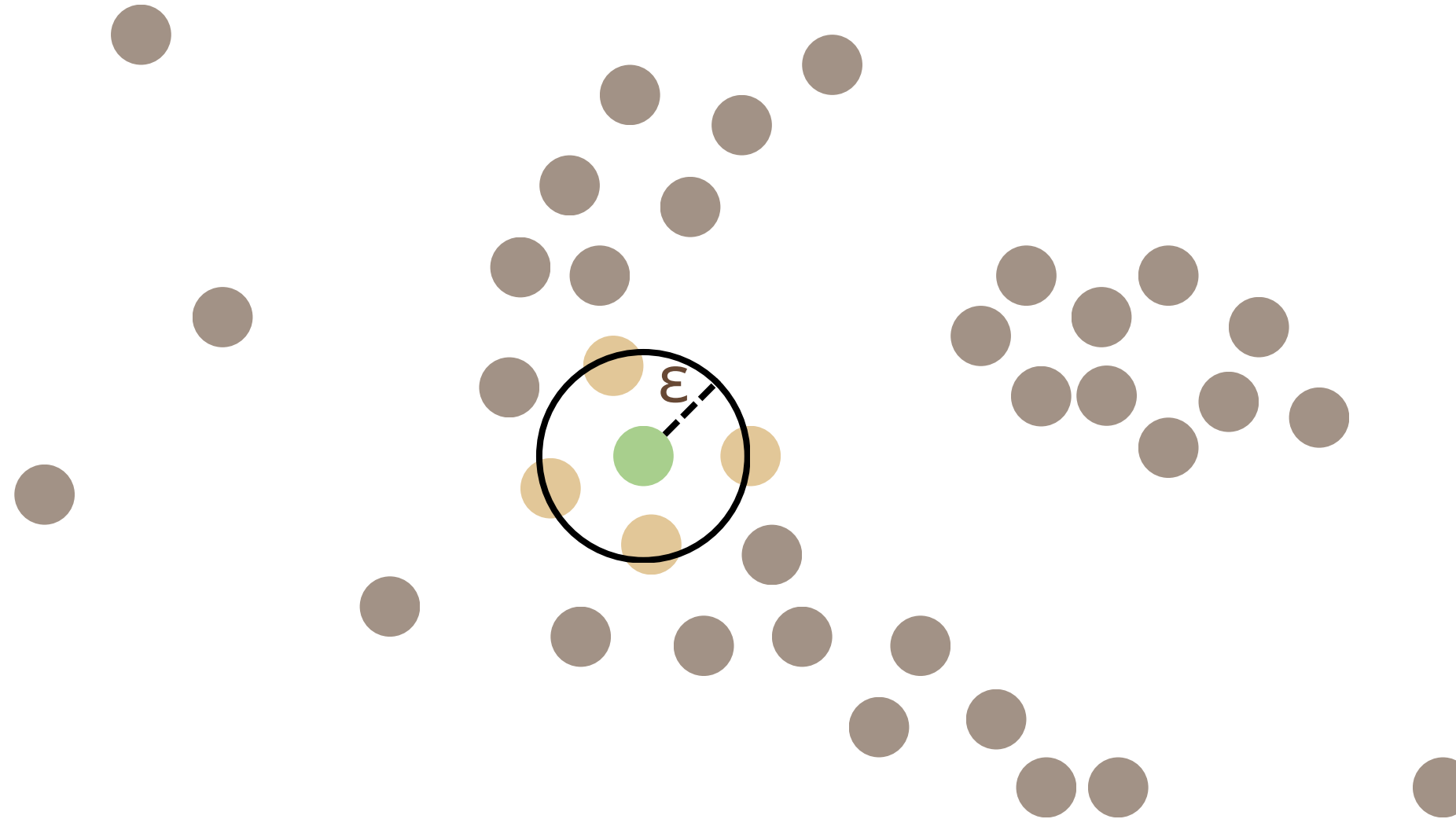
minPts = 4



Primero contamos la cantidad de puntos más cercanos a cada punto, si la cantidad de puntos es mayor a igual a **minPts** entonces ese es un **Core Point**

DBSCAN

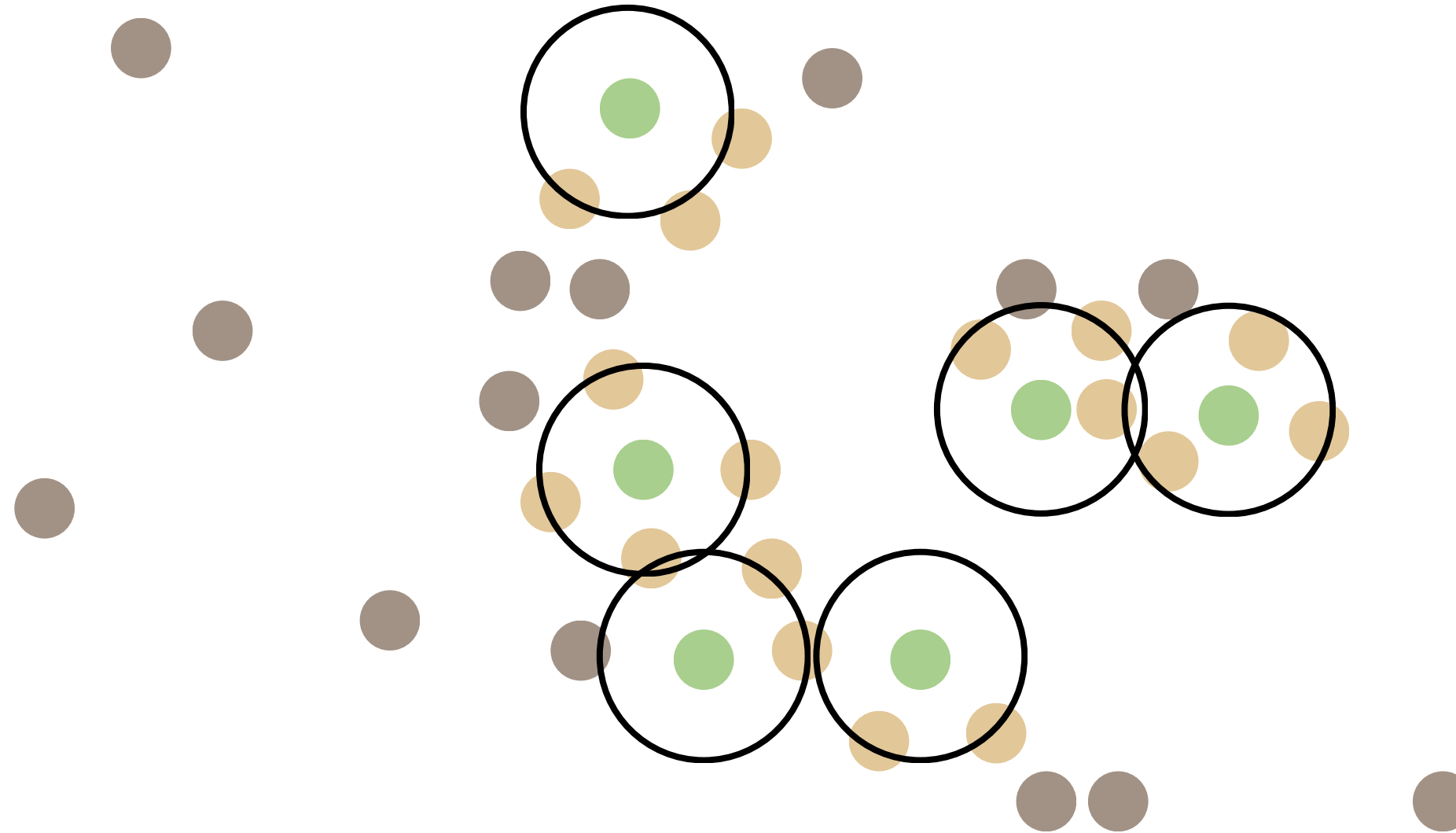
minPts = 4



Primero contamos la cantidad de puntos más cercanos a cada punto, si la cantidad de puntos es mayor a igual a **minPts** entonces ese es un **Core Point**

DBSCAN

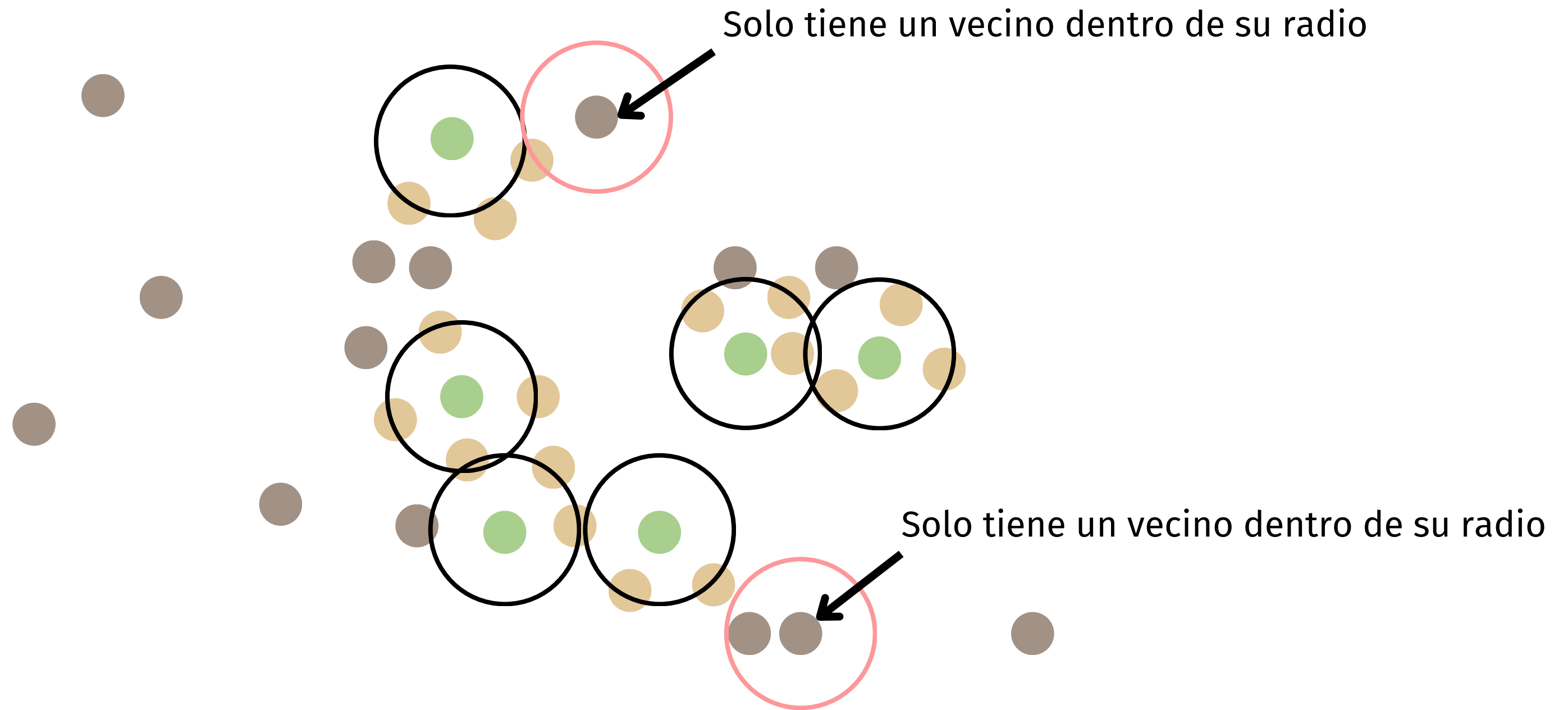
minPts = 4



Primero contamos la cantidad de puntos más cercanos a cada punto, si la cantidad de puntos es mayor a igual a **minPts** entonces ese es un **Core Point**

DBSCAN

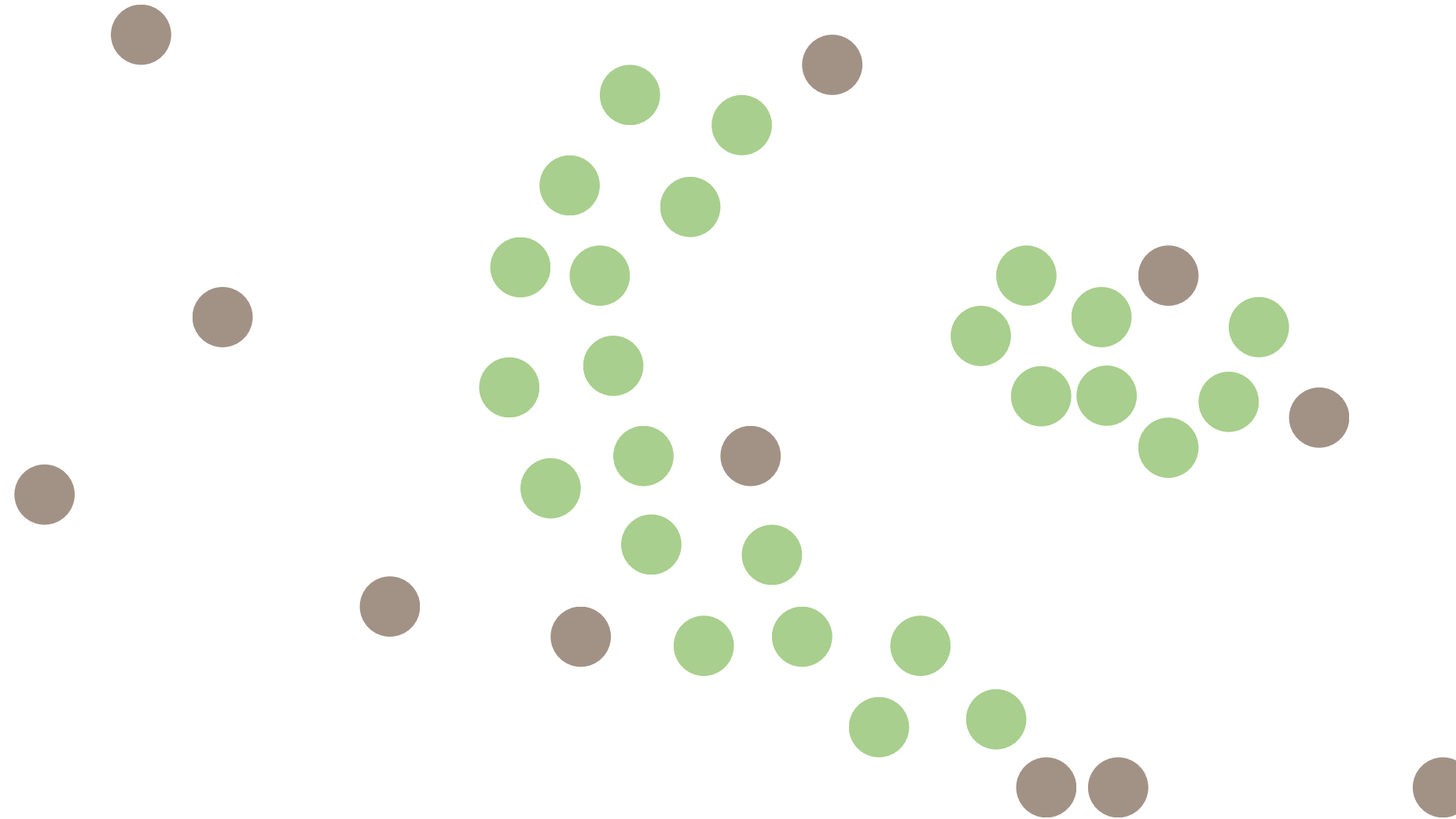
minPts = 4



Primero contamos la cantidad de puntos más cercanos a cada punto, si la cantidad de puntos es mayor a igual a **minPts** entonces ese es un **Core Point**

DBSCAN

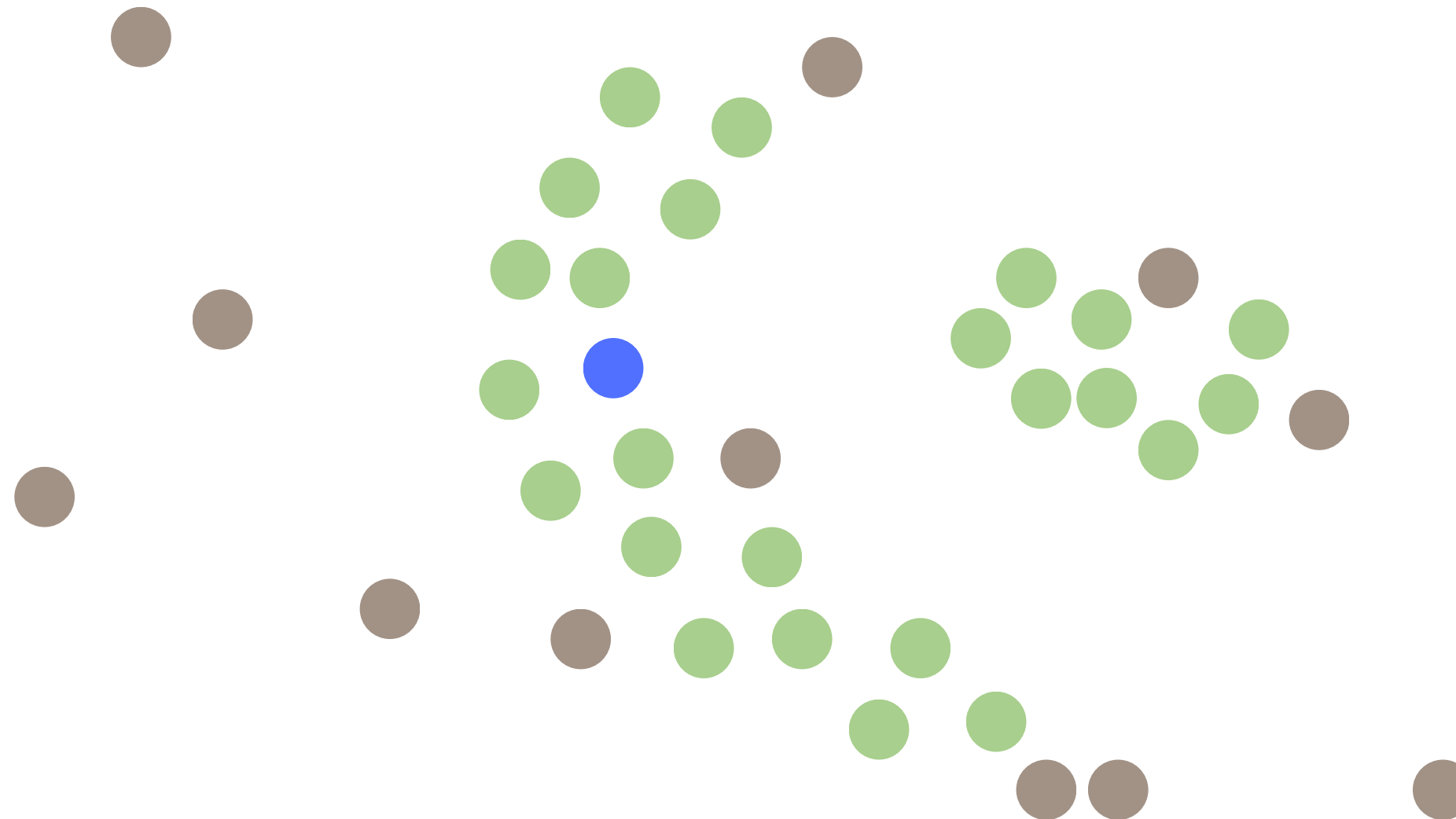
minPts = 4



Los puntos verdes son todos los **Core Points**, estos puntos todavía están asignados a ningún cluster, solo cumplen con el criterio de tener al menos **MinPts** vecinos

DBSCAN

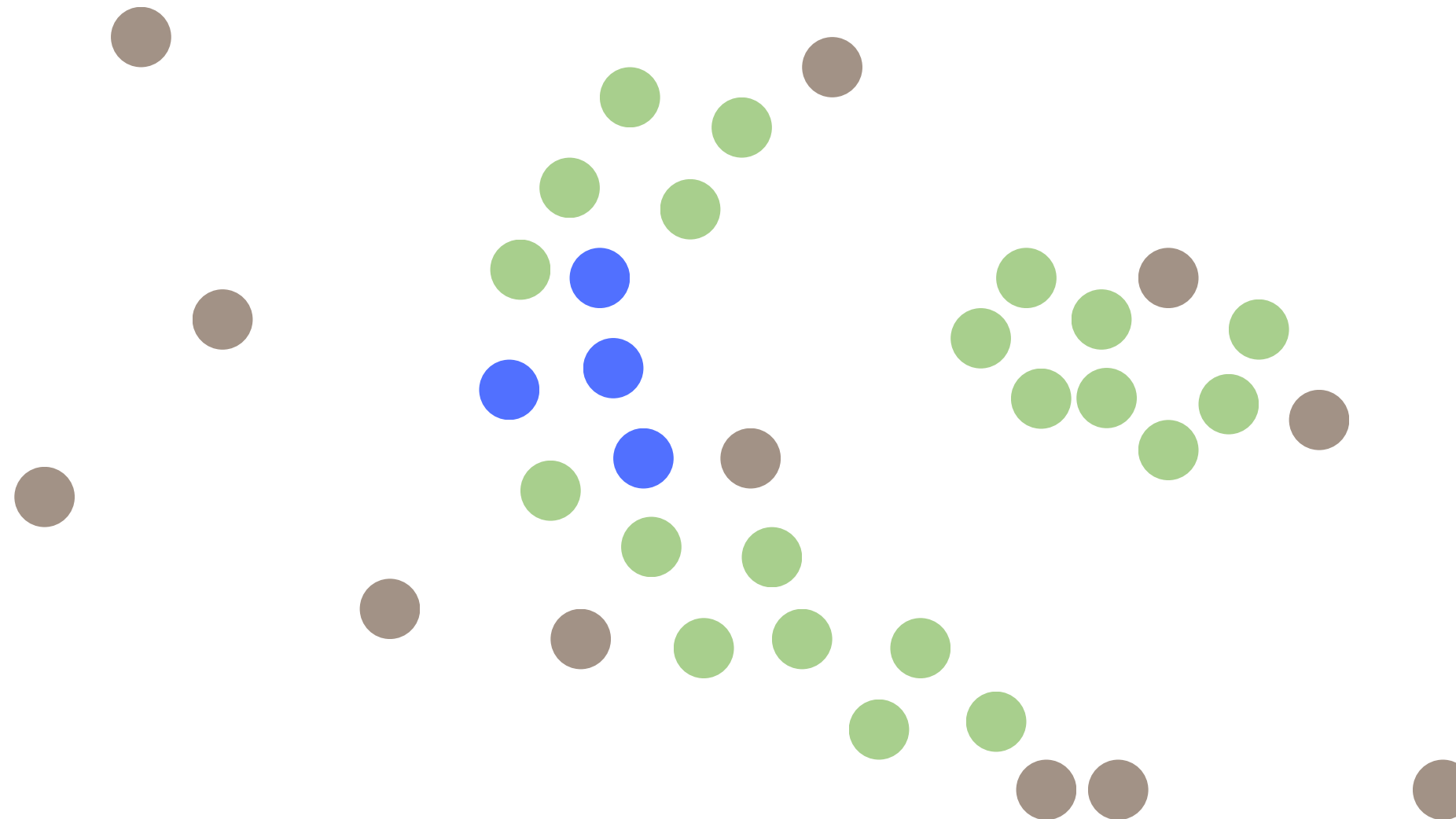
minPts = 4



Escogemos un **Core Point** al azar, ese va a ser el primer punto del **primer cluster**.
Desde ese punto, vamos a agregar al cluster los puntos “core” cercanos a cualquier punto que ya esté dentro del cluster

DBSCAN

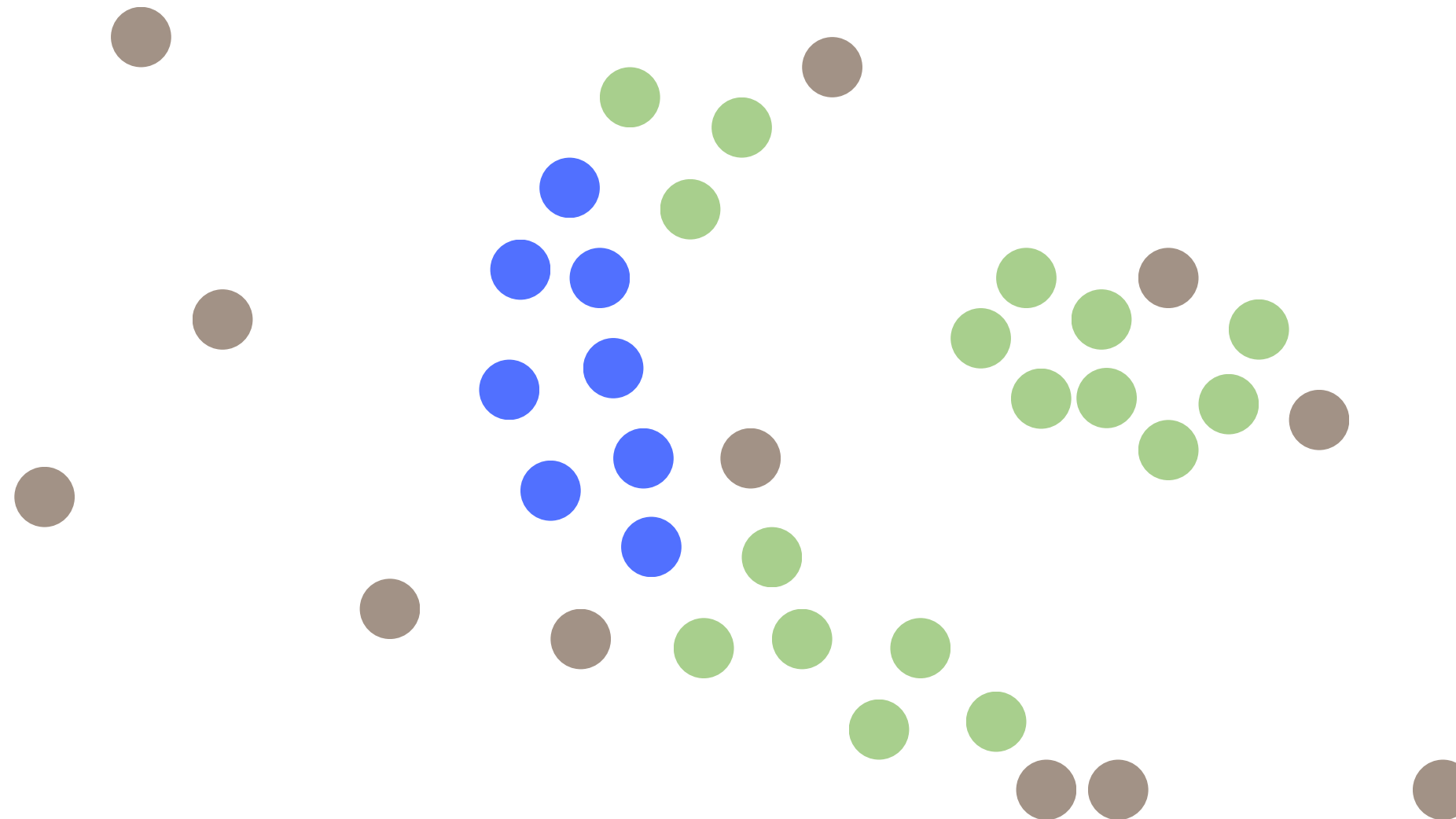
minPts = 4



Escogemos un **Core Point** al azar, ese va a ser el primer punto del **primer cluster**.
Desde ese punto, vamos a agregar al cluster los puntos “core” cercanos a cualquier punto que ya esté dentro del cluster

DBSCAN

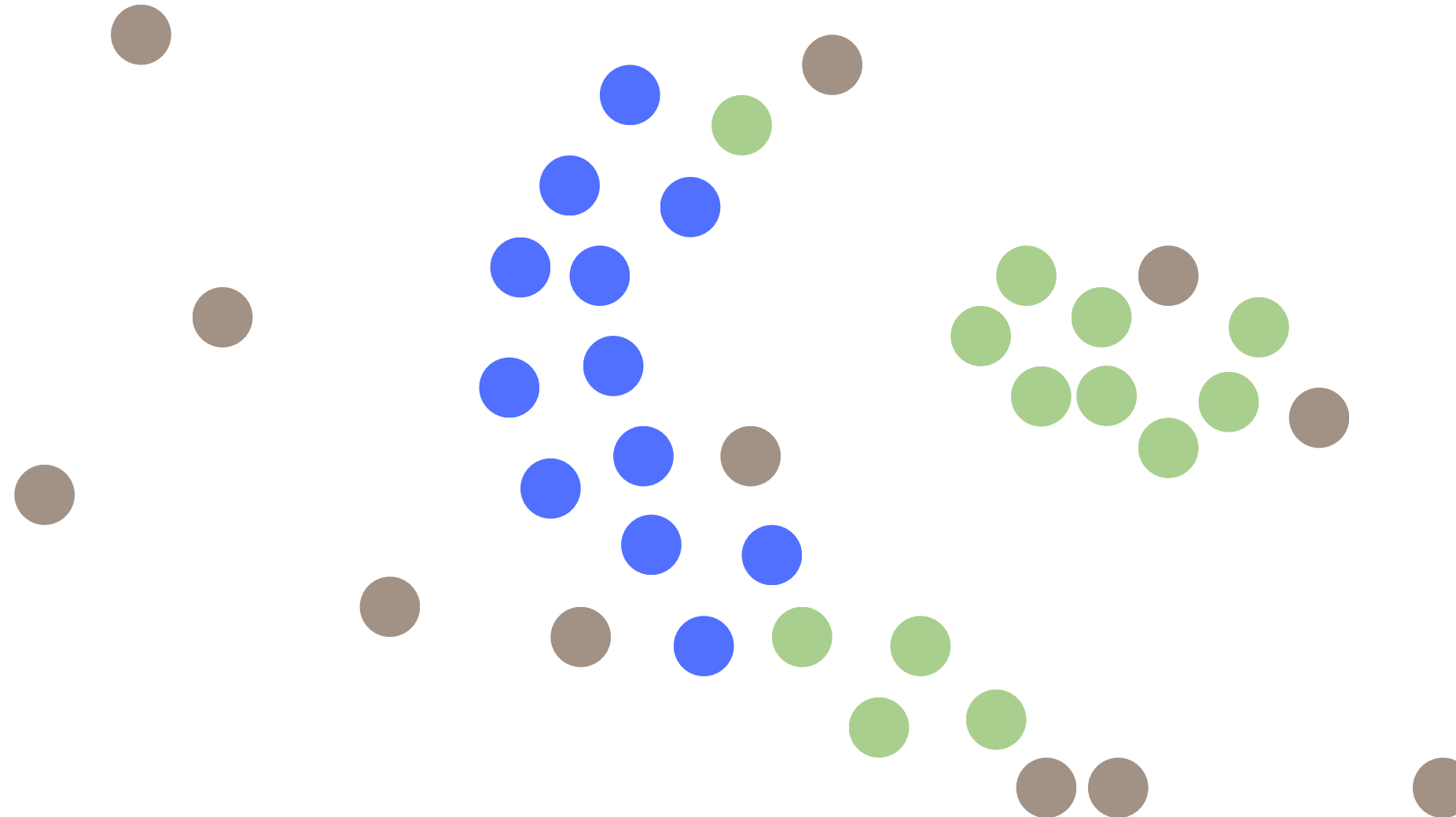
minPts = 4



Escogemos un **Core Point** al azar, ese va a ser el primer punto del **primer cluster**.
Desde ese punto, vamos a agregar al cluster los puntos “core” cercanos a cualquier
punto que ya esté dentro del cluster

DBSCAN

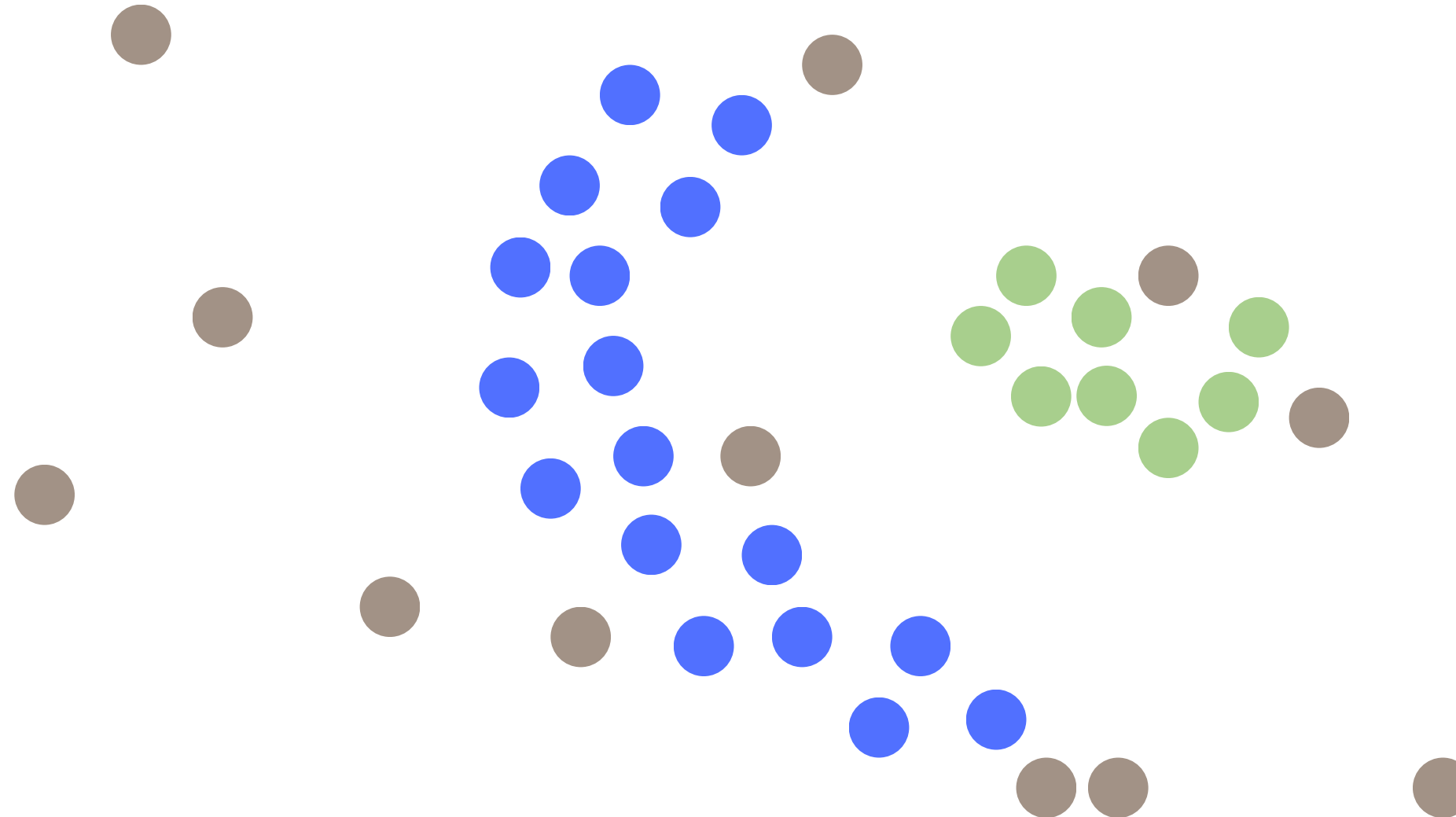
minPts = 4



Escogemos un **Core Point** al azar, ese va a ser el primer punto del **primer cluster**.
Desde ese punto, vamos a agregar al cluster los puntos “core” cercanos a cualquier punto que ya esté dentro del cluster

DBSCAN

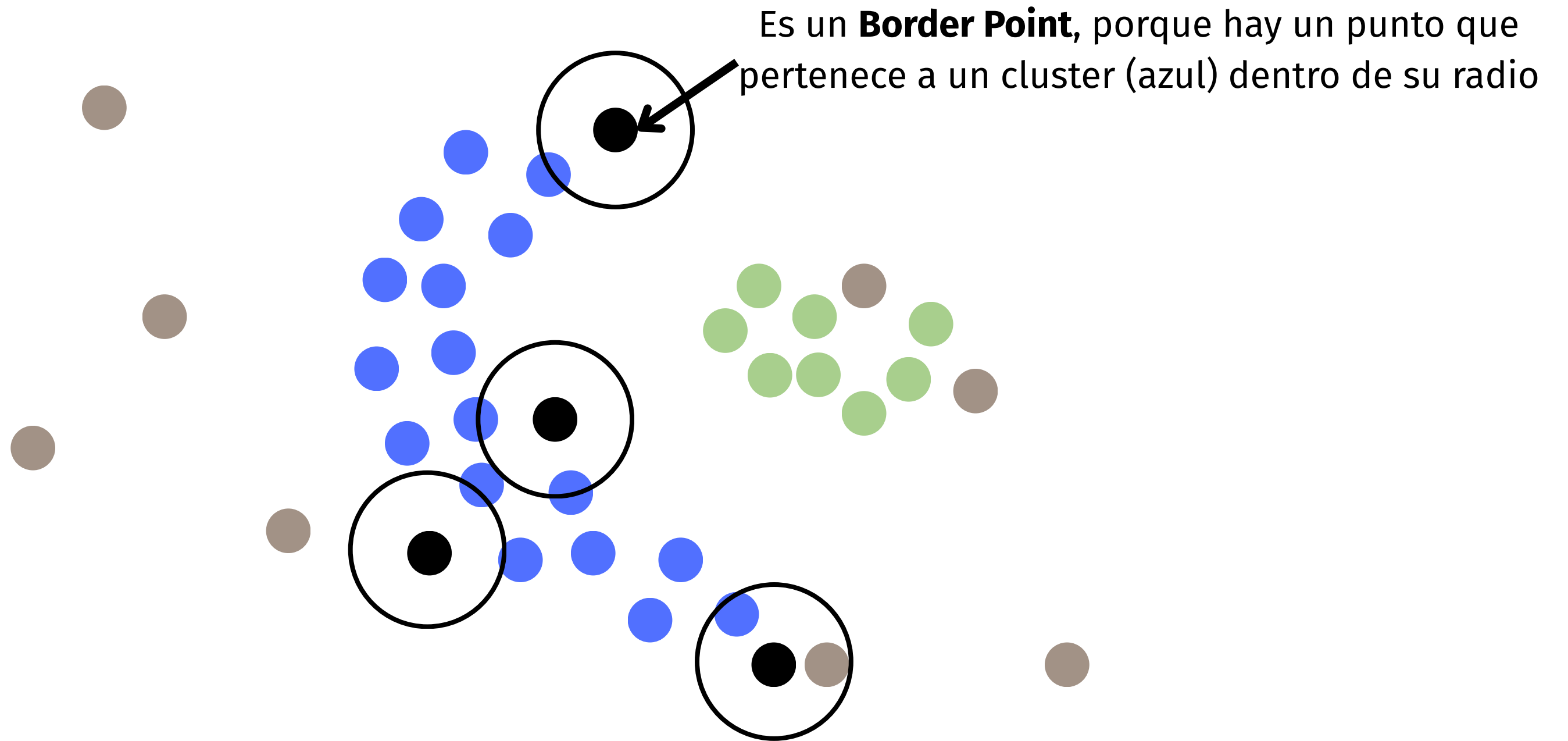
minPts = 4



Escogemos un **Core Point** al azar, ese va a ser el primer punto del **primer cluster**.
Desde ese punto, vamos a agregar al cluster los puntos “core” cercanos a cualquier punto que ya esté dentro del cluster

DBSCAN

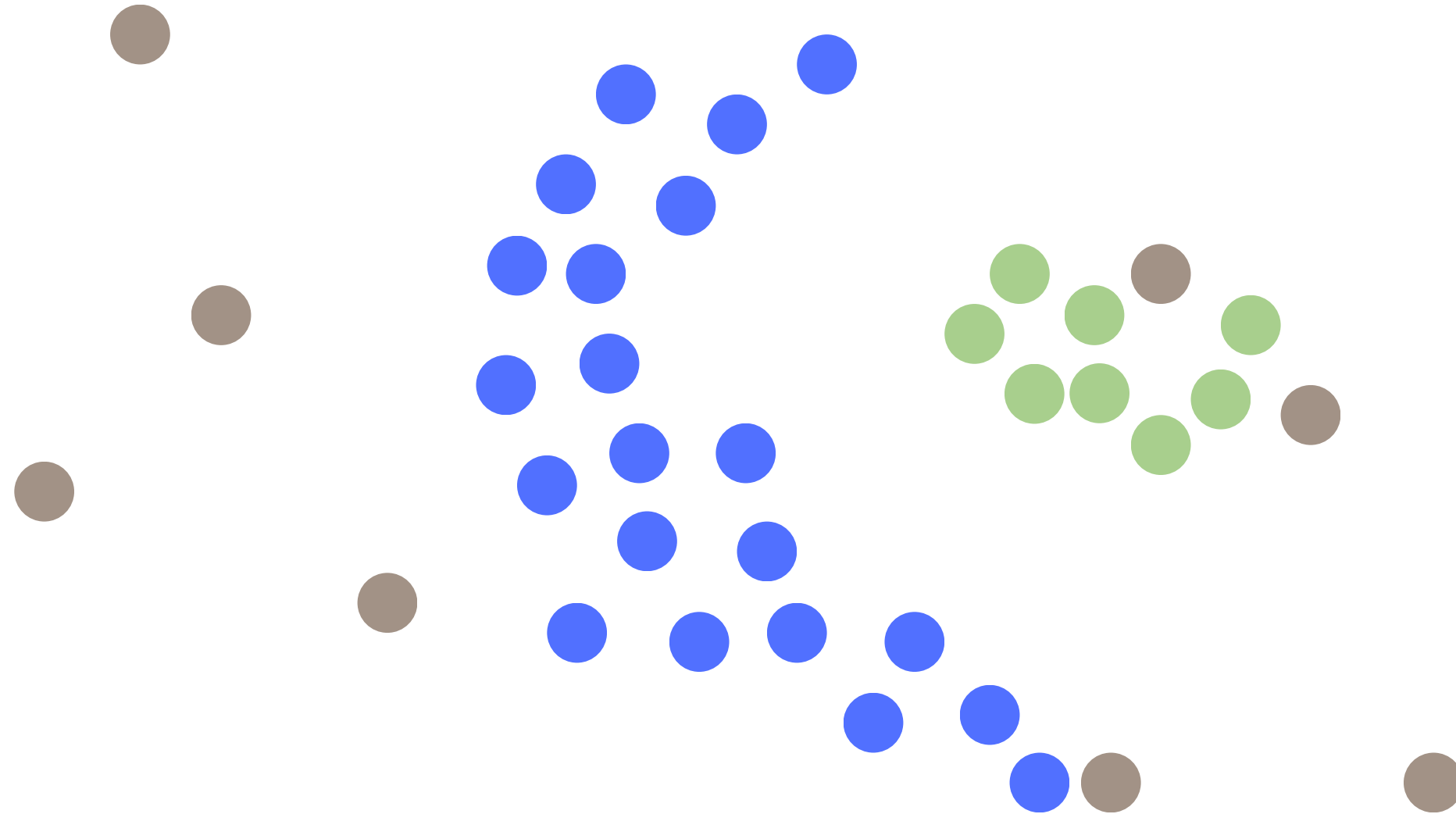
minPts = 4



Se agregan todos los Border Point al cluster

DBSCAN

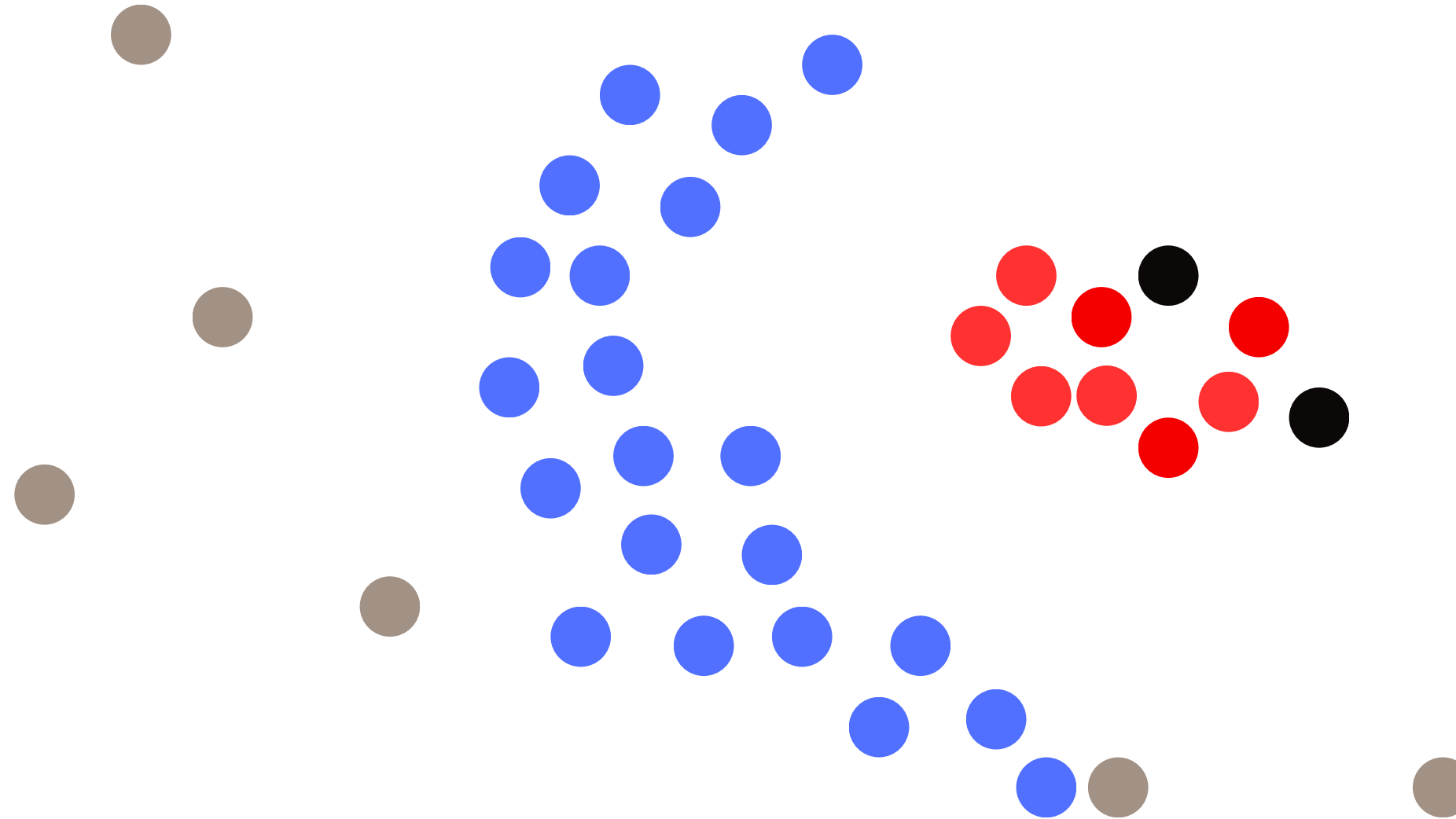
minPts = 4



Se agregan todos los Border Point al cluster

DBSCAN

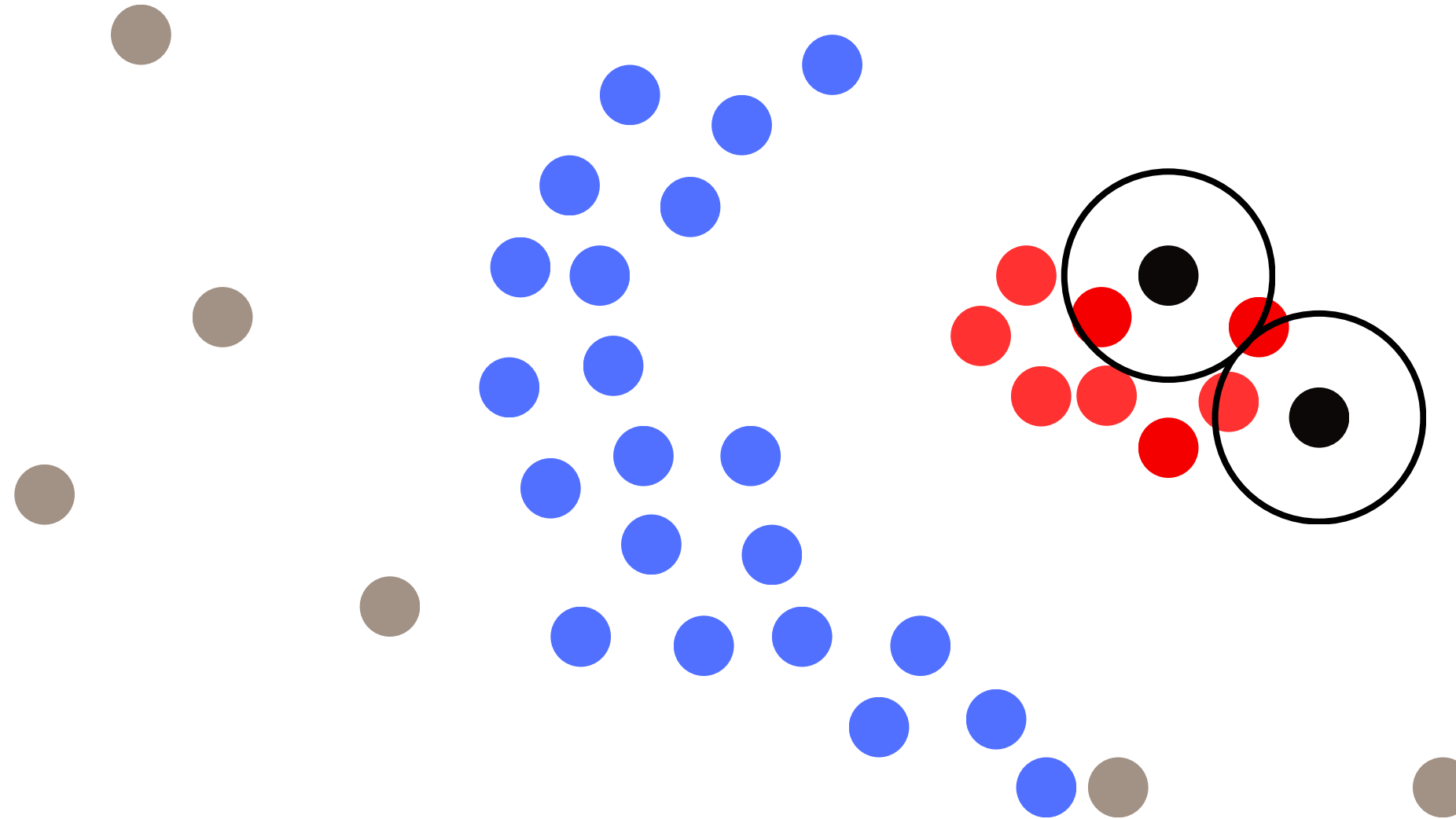
minPts = 4



Una vez que ya no se pueden agregar más puntos al cluster, se escoge otro **Core Point** al azar para repetir el proceso

DBSCAN

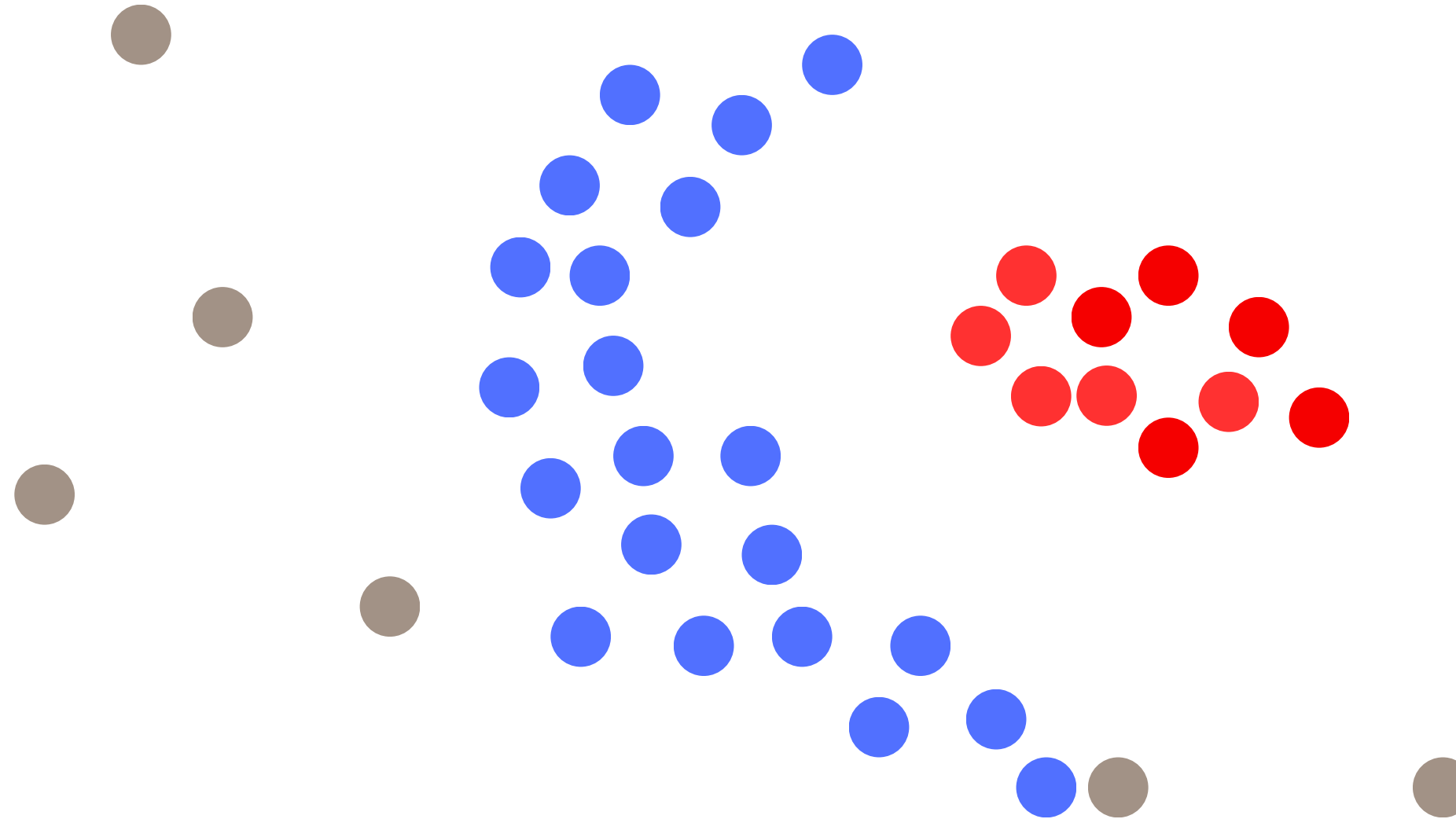
minPts = 4



Una vez que ya no se pueden agregar más puntos al cluster, se escoge otro **Core Point** al azar para repetir el proceso

DBSCAN

minPts = 4



Finalmente nos quedamos con dos cluster, sin definir previamente la cantidad de clusters (k). Los puntos restantes son outliers

Gráfico k-dist

El k-dist plot nos permite **definir el valor de ϵ** :

1. Para cada punto, calcular la distancia al minPts-ésimo vecino más cercano.
2. Ordenar esas distancias de menor a mayor.
3. El valor de ϵ se elige en la zona donde aparece un codo en la curva

Aprox. 1000 nodos tienen distancia = 5
a su cuarto vecino

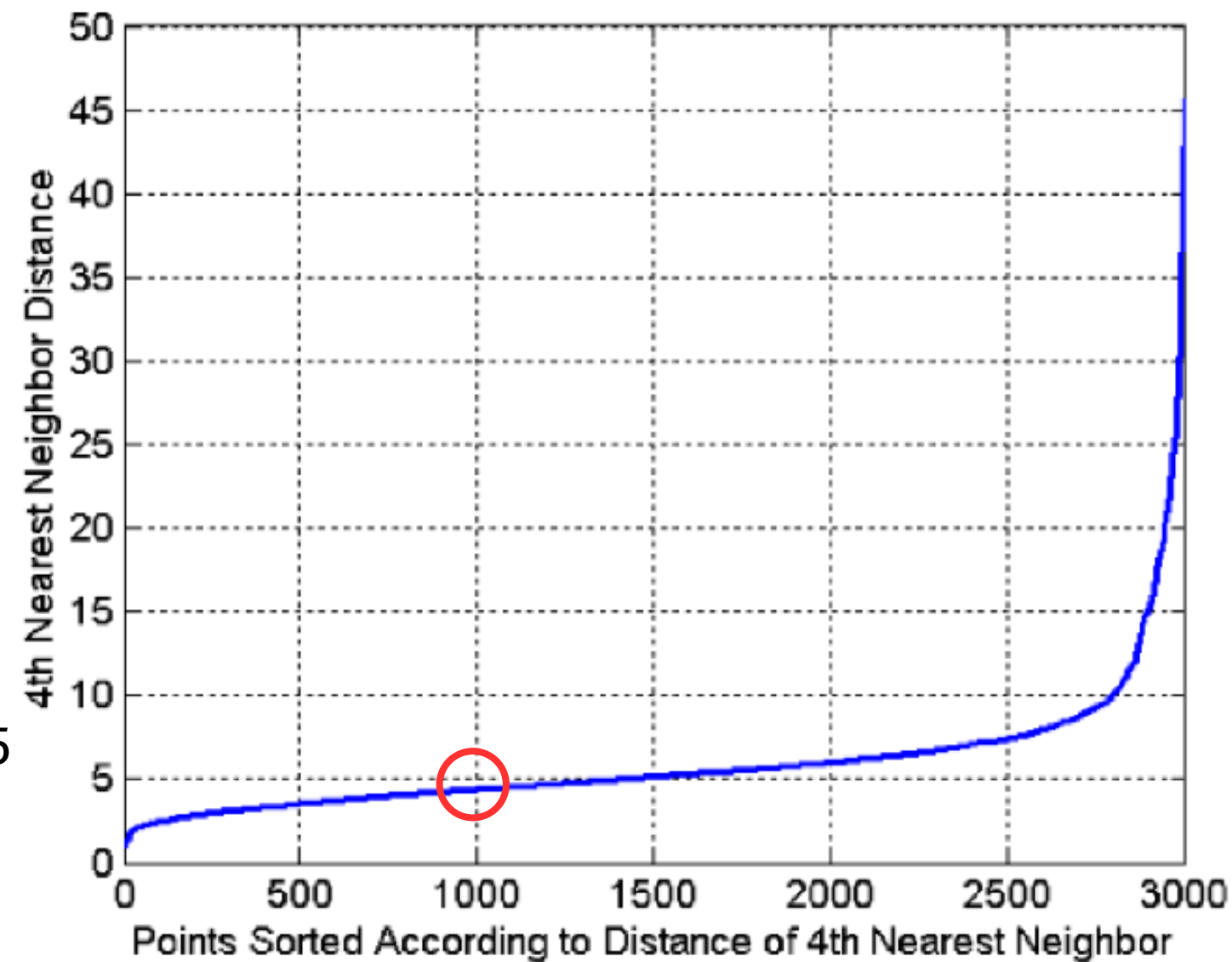
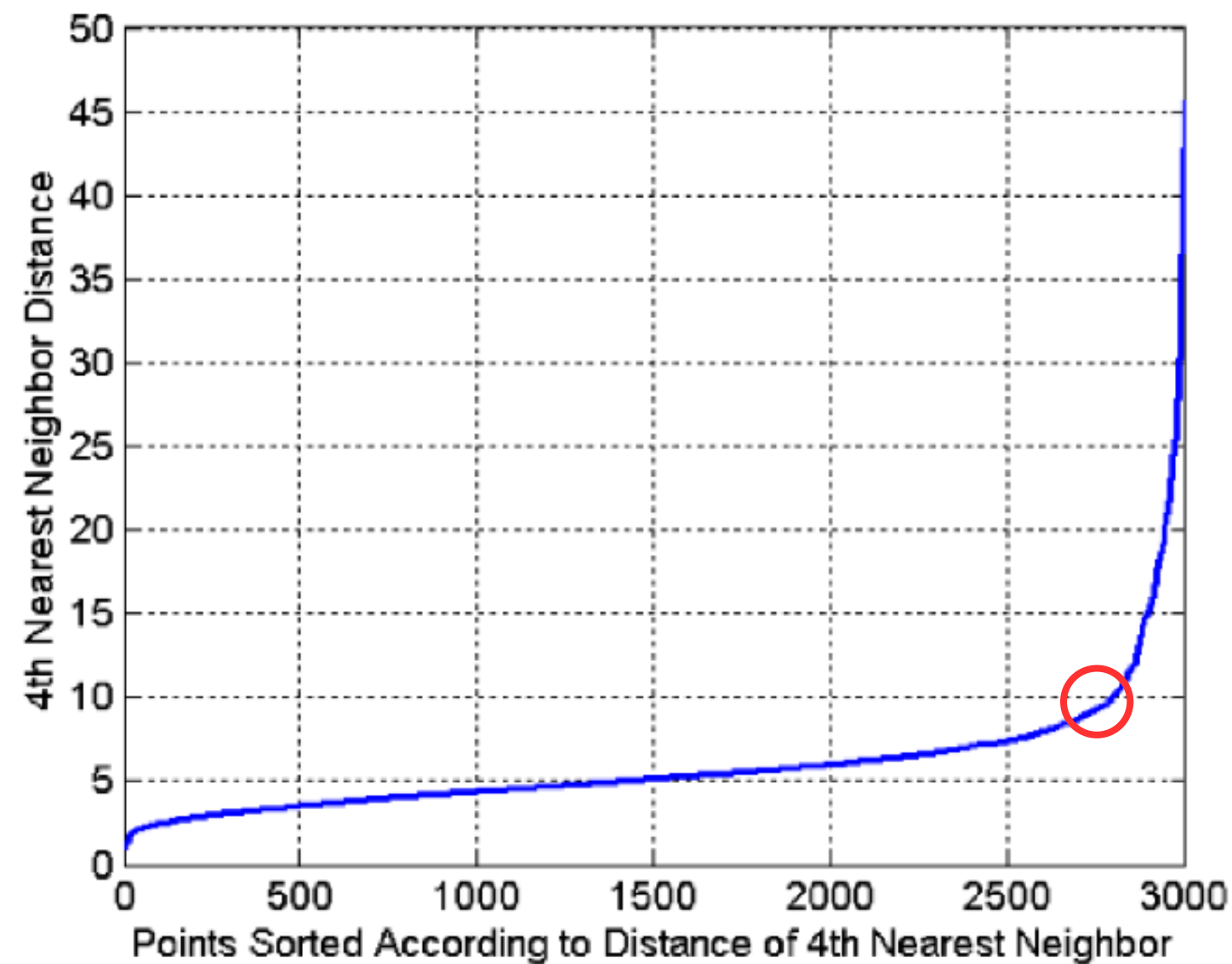


Gráfico k-dist

El k-dist plot nos permite **definir el valor de ϵ** :

1. Para cada punto, calcular la distancia al minPts-ésimo vecino más cercano.
2. Ordenar esas distancias de menor a mayor.
3. El valor de ϵ se elige en la zona donde aparece un codo en la curva



ϵ pequeños $\rightarrow$ solo núcleos muy densos
 ϵ grandes $\rightarrow$ clusters muy amplios

DBSCAN

```
from sklearn.neighbors import NearestNeighbors
from sklearn.cluster import DBSCAN
from collections import Counter

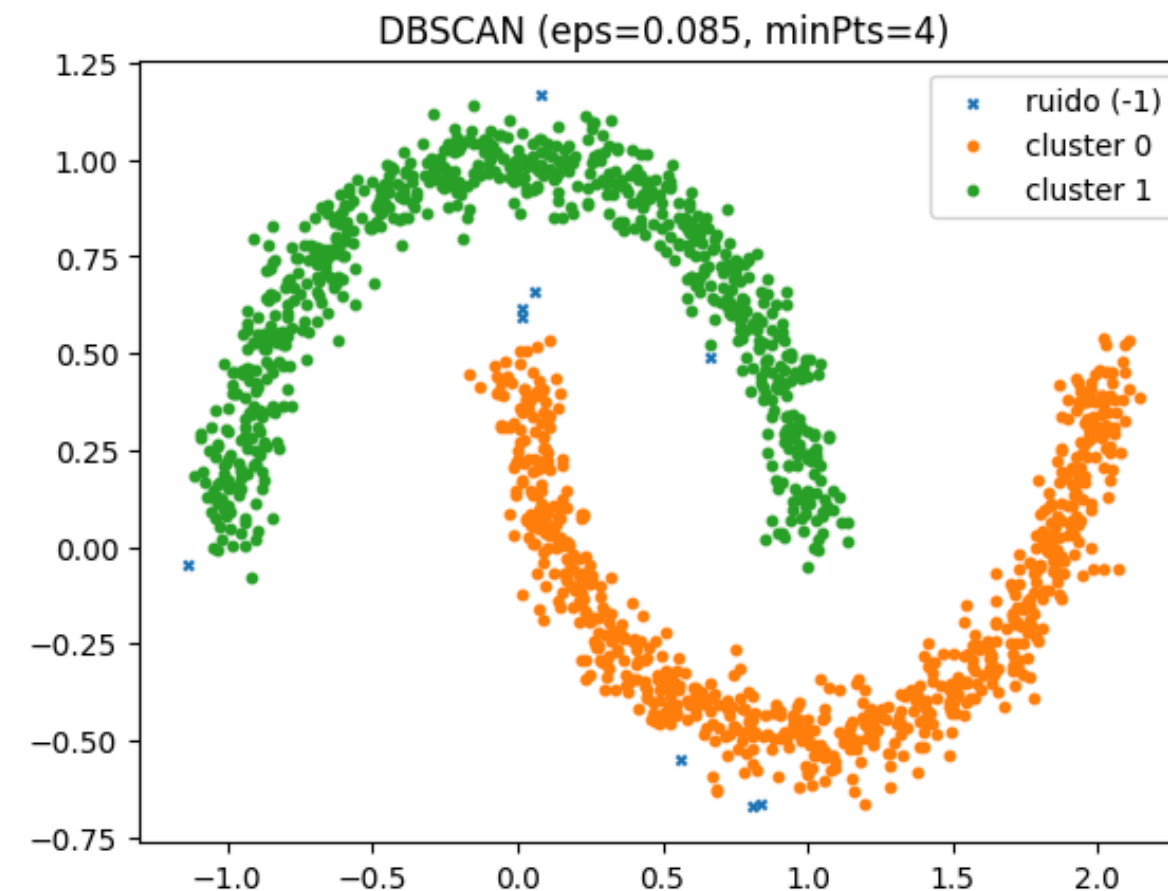
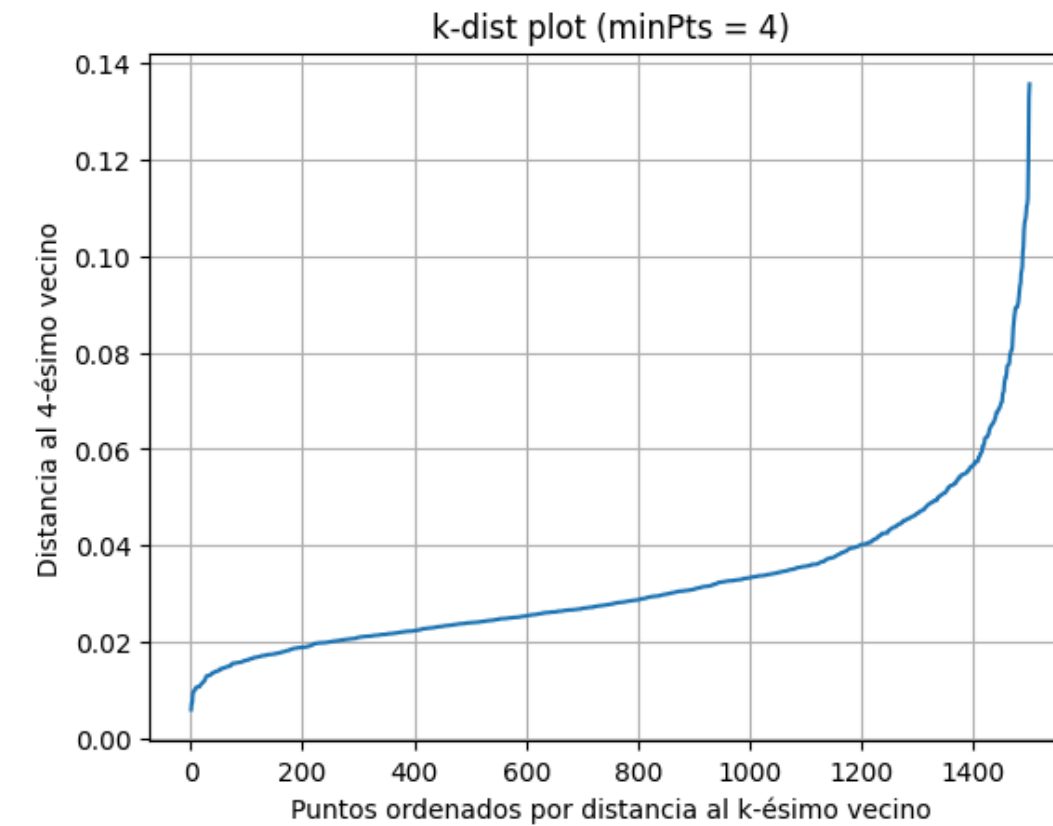
X, _ = make_moons(n_samples=1500, noise=0.06)

# Elegir el "eps"
minPts = 4

# Distancias al k-ésimo vecino más cercano
nn = NearestNeighbors(n_neighbors=minPts).fit(X)
distances, _ = nn.kneighbors(X)
# La distancia relevante es la del minPts vecino
k_dists = np.sort(distances[:, -1])
...
plt.show()

# DBSCAN
db = DBSCAN(eps=best_eps, min_samples=minPts)
labels = db.fit_predict(X)

# Para graficar
counts = Counter(labels)
print("Tamaño por cluster (label -> n):", dict(counts))
print("Ruido (label -1):", counts.get(-1, 0))
...
plt.show()
```



Adjusted Rand Index

El **Adjusted Rand Index (ARI)** mide qué tan similares son dos particiones de los mismos datos. A diferencia de Silhouette o Elbow, el ARI es una métrica externa: **necesita ground truth**, no evalúa la estructura interna del clustering por sí sola.

Punto de partida: el Rand Index (RI)

Dadas dos particiones de **n** puntos, se cuentan cuatro tipos de pares de puntos:

- **a** = pares que están juntos en ambas particiones
- **b** = pares que están separados en ambas particiones
- **c, d** = pares que coinciden en una partición pero no en la otra (desacuerdos)

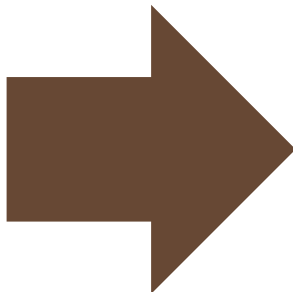
$$RI = \frac{a + b}{\binom{n}{2}}$$

Ejemplo de RI

Supongamos 5 elementos. Tenemos dos "particiones":

- P1 (verdad real): A, A, B, B, B
- P2 (lo que hizo un algoritmo): Cluster 1, Cluster 1, Cluster 1, Cluster 2, Cluster 2

Puntos	True Label (GT)	Cluster Asig. (CA)
P1	A	1
P2	A	1
P3	B	1
P4	B	2
P5	B	2



Pareja	¿Junta en GT?	¿Junta en CA?	Tipo
(P1,P2)	Sí (A,A)	Sí (1,1)	a
(P1,P3)	No (A,B)	Sí (1,1)	d
(P1,P4)	No (A,B)	No (1,2)	b
...	...	...	
(P3,P5)	Sí (B,B)	No (1,2)	c
(P4,P5)	Sí (B,B)	Sí (2,2)	a

- a=2 (parejas que ambas particiones ponen juntas)
- b=4 (parejas que ambas particiones ponen separadas)

Adjusted Rand Index

El problema del Rand Index: incluso una asignación completamente aleatoria de clusters obtiene un RI relativamente alto, porque hay muchos pares que "coinciden por azar" simplemente al estar separados. Esto infla el score y lo hace difícil de interpretar.

Corrección

$$ARI = \frac{RI - E[RI]}{\max(RI) - E[RI]}$$

donde $E[RI]$ es el valor esperado del Rand Index bajo asignación aleatoria (calculado con hipergeométrica sobre las tablas de contingencia).

Interpretación

- **ARI = 1:** las dos particiones coinciden perfectamente
- **ARI $\approx$ 0:** el clustering es equivalente a una asignación aleatoria
- **ARI < 0:** el clustering es peor que aleatorio

Adjusted Rand Index

```
from sklearn.metrics import adjusted_rand_score, rand_score

# Label real (ground truth) vs. clusters que asignó el algoritmo
label_real = ['A', 'A', 'B', 'B', 'B']
cluster_algoritmo = [1, 1, 1, 2, 2]

ri = rand_score(label_real, cluster_algoritmo)
ari = adjusted_rand_score(label_real, cluster_algoritmo)

print(f"Rand Index: {ri:.3f}")
print(f"Adjusted Rand Index: {ari:.3f}")

Rand Index: 0.600
Adjusted Rand Index: 0.167
```

El **RI=0.6** se ve como un score decente, pero gran parte de ese acuerdo era esperable incluso sin que el algoritmo hiciera nada inteligente, solo por los tamaños de los grupos. El **ARI** corrige eso y da 0.167, un número mucho más honesto sobre qué tan bien el algoritmo recuperó el label real.



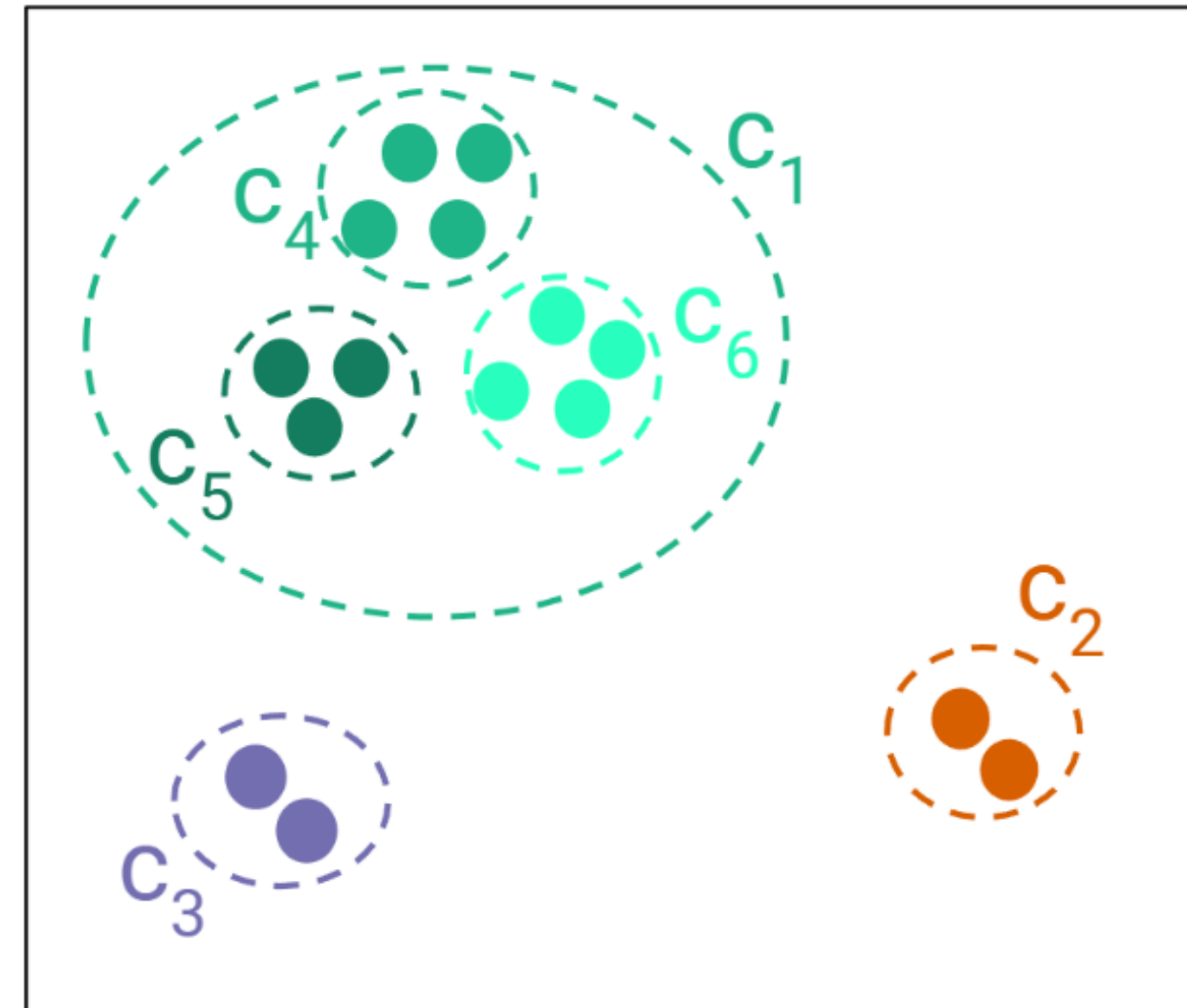
Clustering Jerarquico

Clustering Jerarquico

Es un método de agrupamiento **no supervisado** que busca **organizar objetos en una jerarquía** de grupos o clústeres, según su similitud o distancia.

Construye una jerarquía multinivel de grupos al unir clústeres más pequeños en otros más grandes o dividir un clúster grande en otros más pequeños.

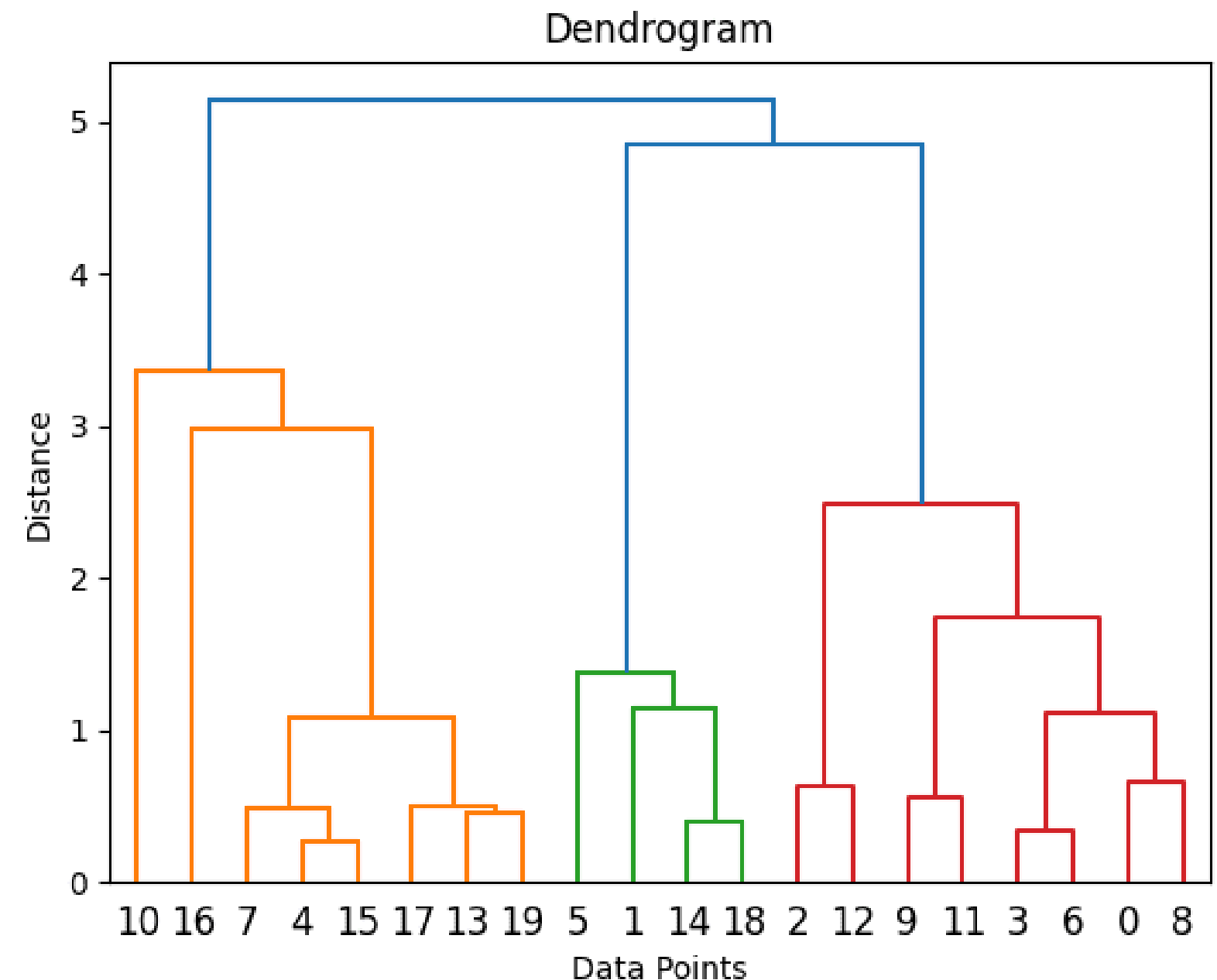
Esto da como resultado una estructura en forma de árbol conocida como **dendrograma**.



Dendrograma

En lugar de asignar cada punto directamente a un grupo (como hace K-Means), el clustering jerárquico construye un **dendrograma**, que muestra cómo **los grupos se van formando o dividiendo paso a paso**.

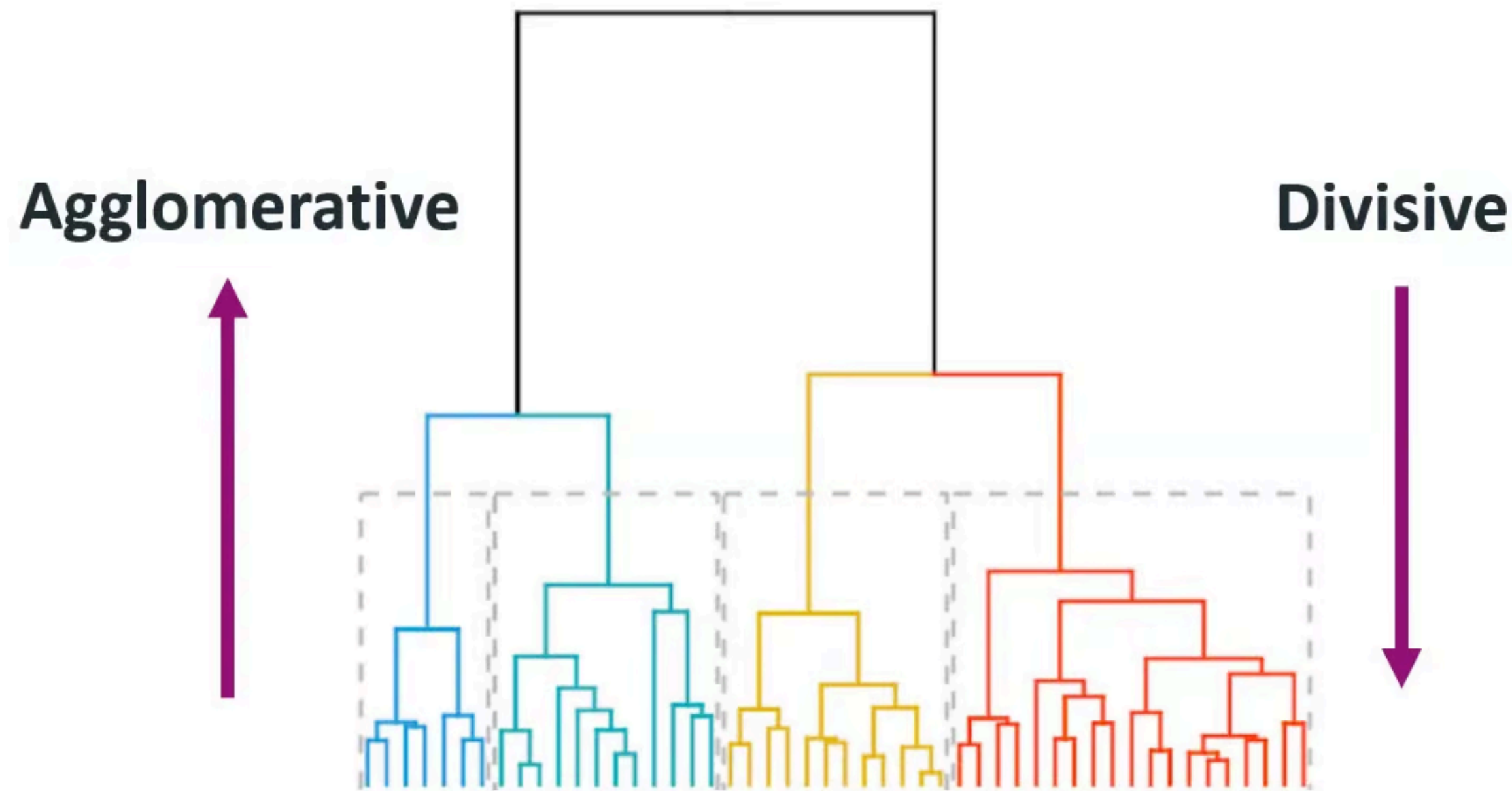
- La altura de las ramas en un dendrograma representa la distancia o disimilitud a la que los clústeres se unen.
- Las alturas más bajas indican clústeres que se combinan a distancias menores, lo que representa una mayor similitud.



Tipos de clustering jerárquico

Aglomerativo (bottom-up): Empieza con cada punto separado y va uniendo los más similares hasta formar un clúster

Divisivo (top-down): Empieza con todos los puntos juntos y va dividiendo el grupo en partes más pequeñas.





HDBSCAN

HDBSCAN

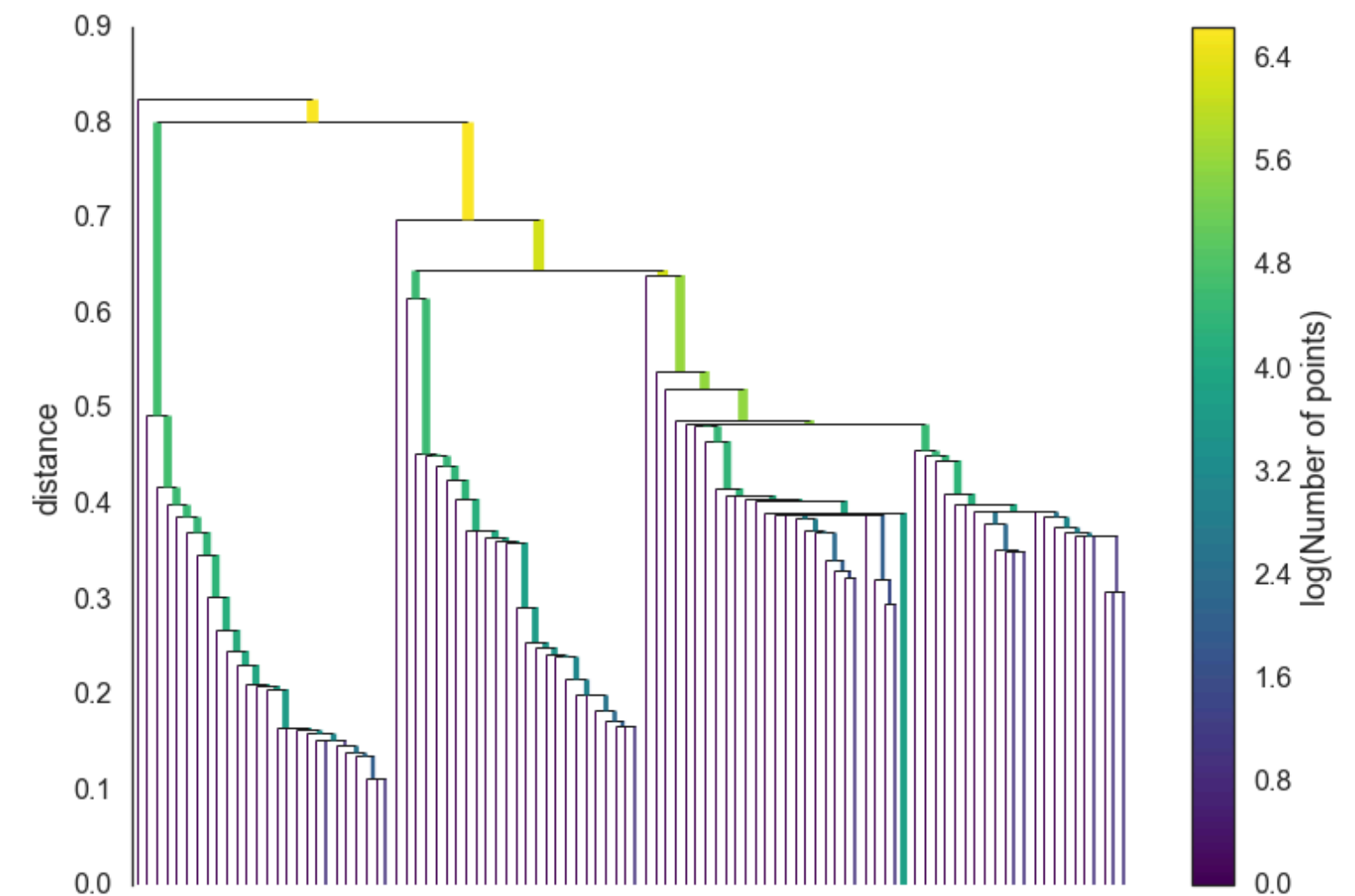
HDBSCAN (Hierarchical Density-Based Spatial Clustering of Applications with Noise) es una versión mejorada de DBSCAN que combina la idea de densidad con la jerarquía de clústeres.

Diferencia con DBSCAN

DBSCAN usa un solo **valor de densidad (eps)** para todo el dataset, mientras que HDBSCAN construye una jerarquía de densidades y selecciona los clústeres más estables.

Ventajas

- Detecta clústeres con **distintas formas y densidades**
- No necesita especificar eps.
- Más robusto al ruido que DBSCAN.
- Produce también una medida de confianza para cada punto.



HDBSCAN

1. Mide la densidad local

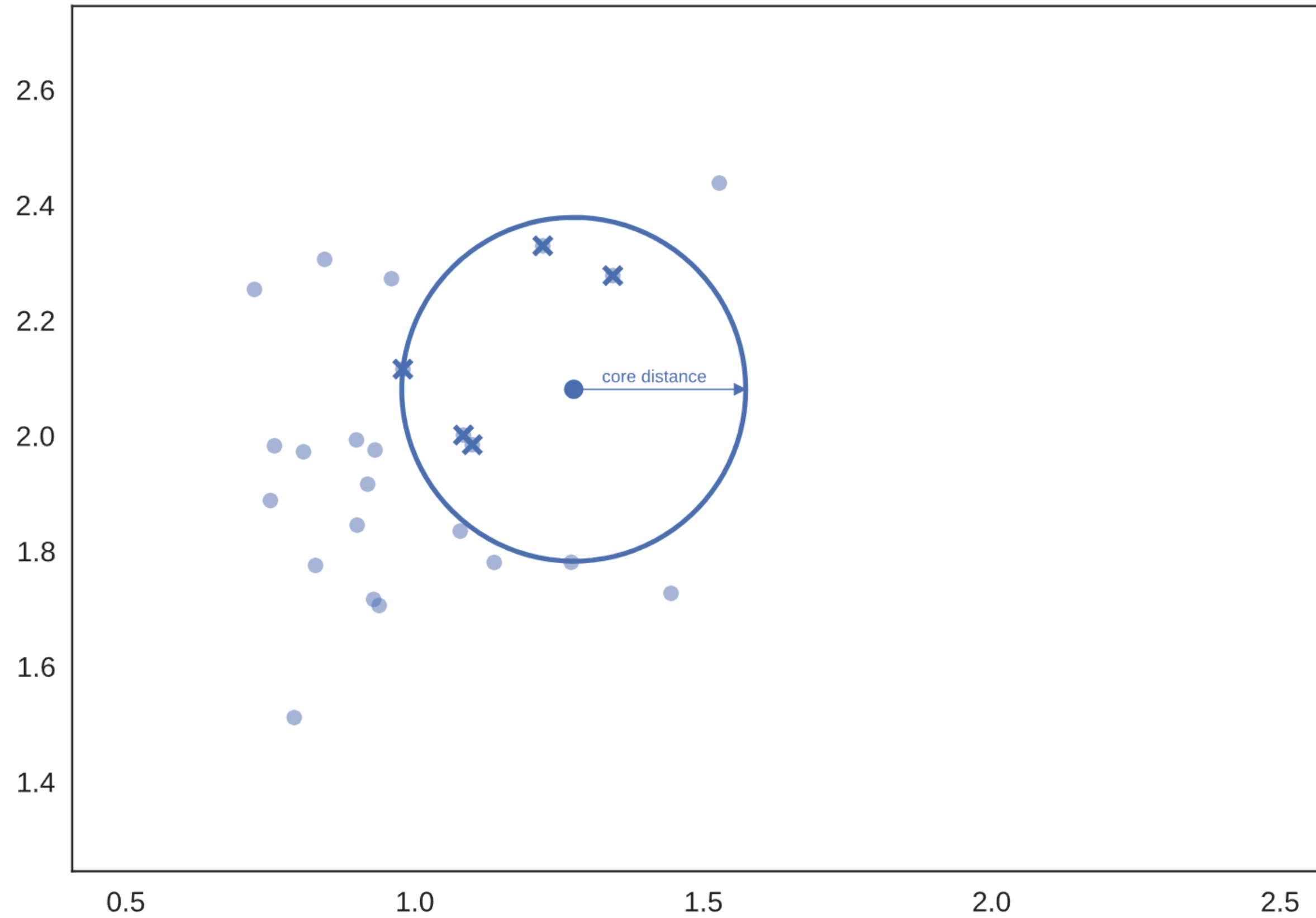
Primero, estima **qué tan denso es el entorno de cada punto**. Para eso, calcula la distancia al vecino número k . Esa distancia se llama distancia de alcance principal (core distance):

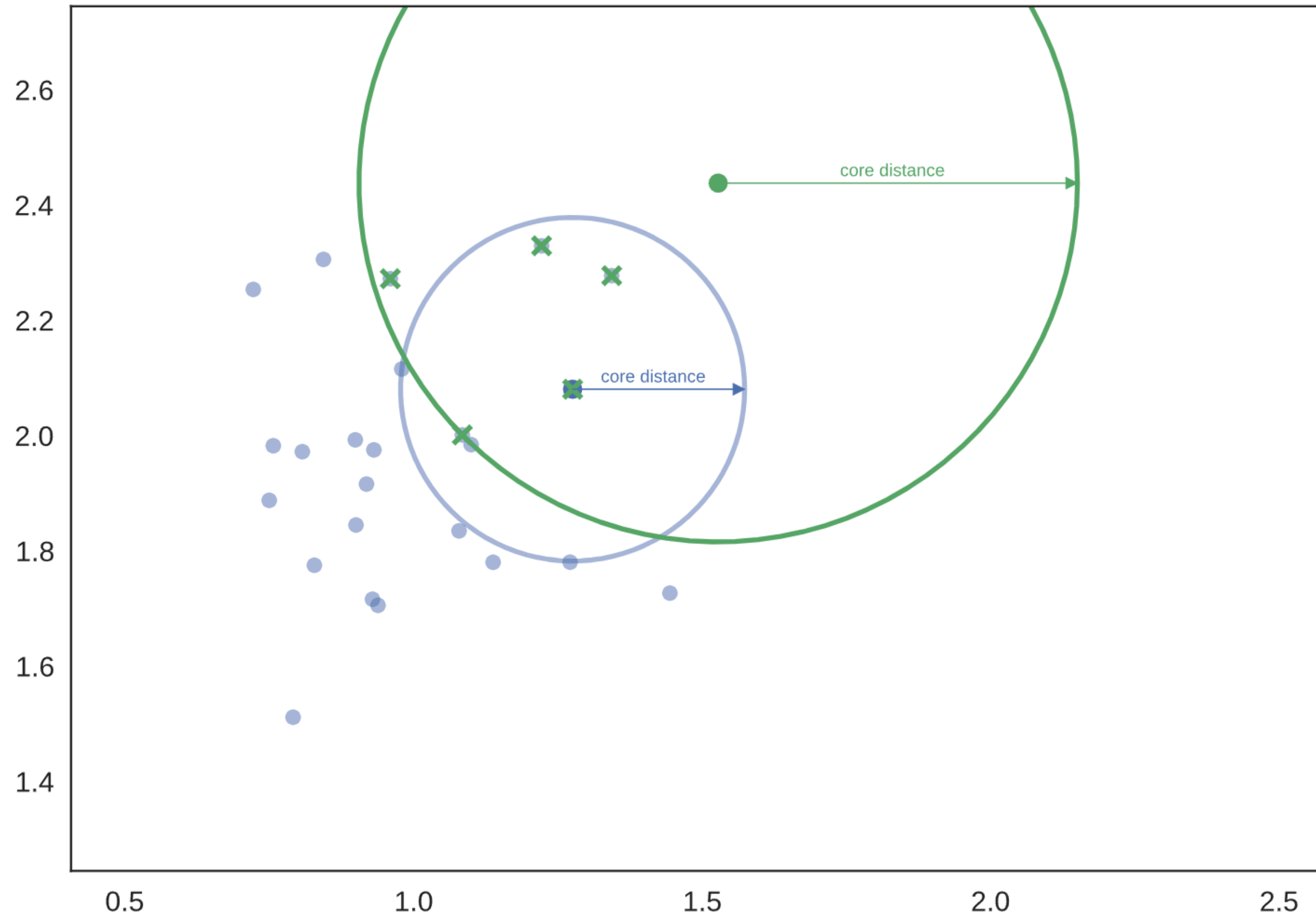
$$core_k(i) = \text{distancia hasta su } k\text{-ésimo vecino mas cercano}$$

2. Mutual reachability distance

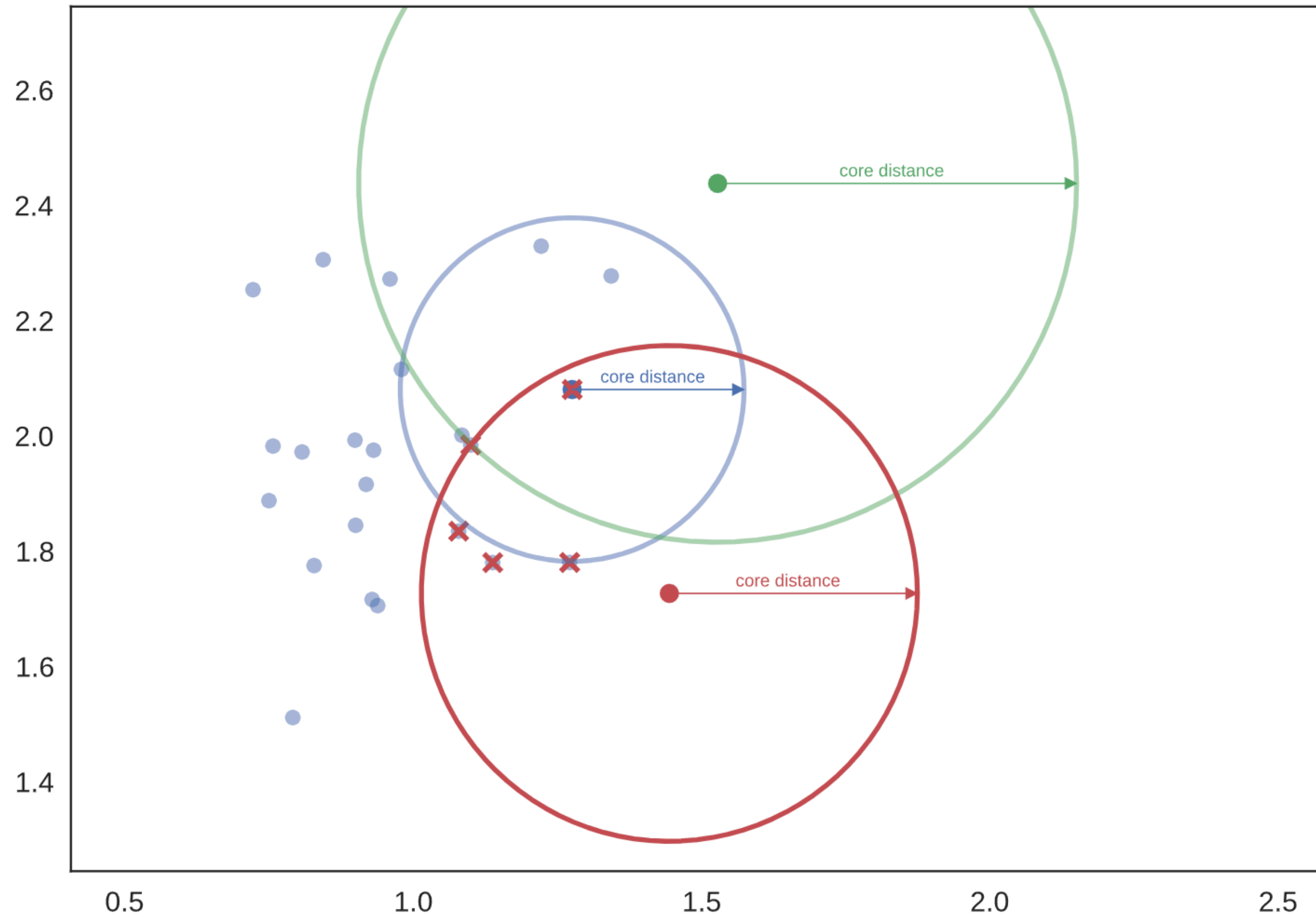
Define la distancia de alcance mutuo entre dos puntos i y j :

$$d_{mreach}(i, j) = \max\{core_k(i), core_k(j), dist(i, j)\}$$

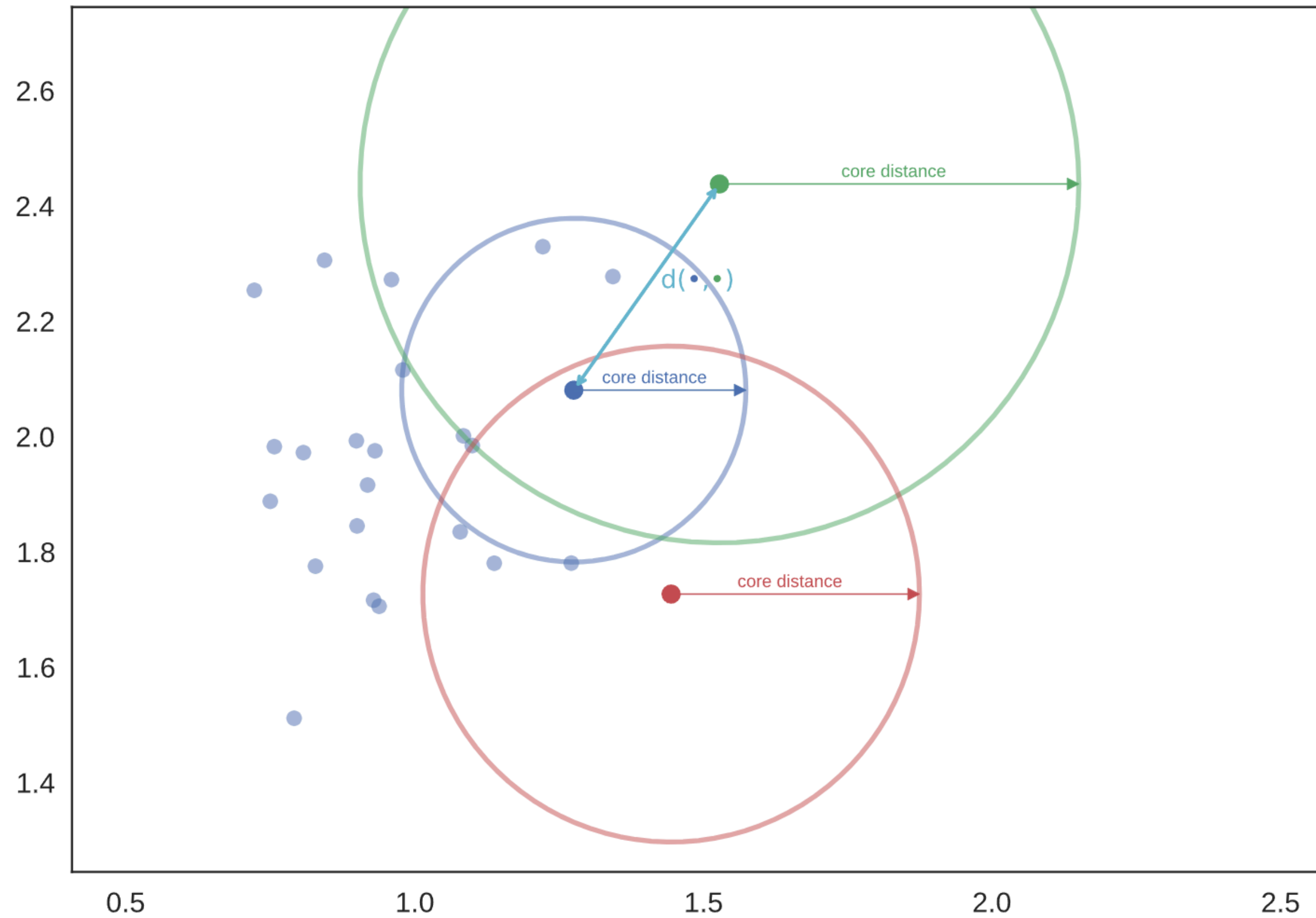




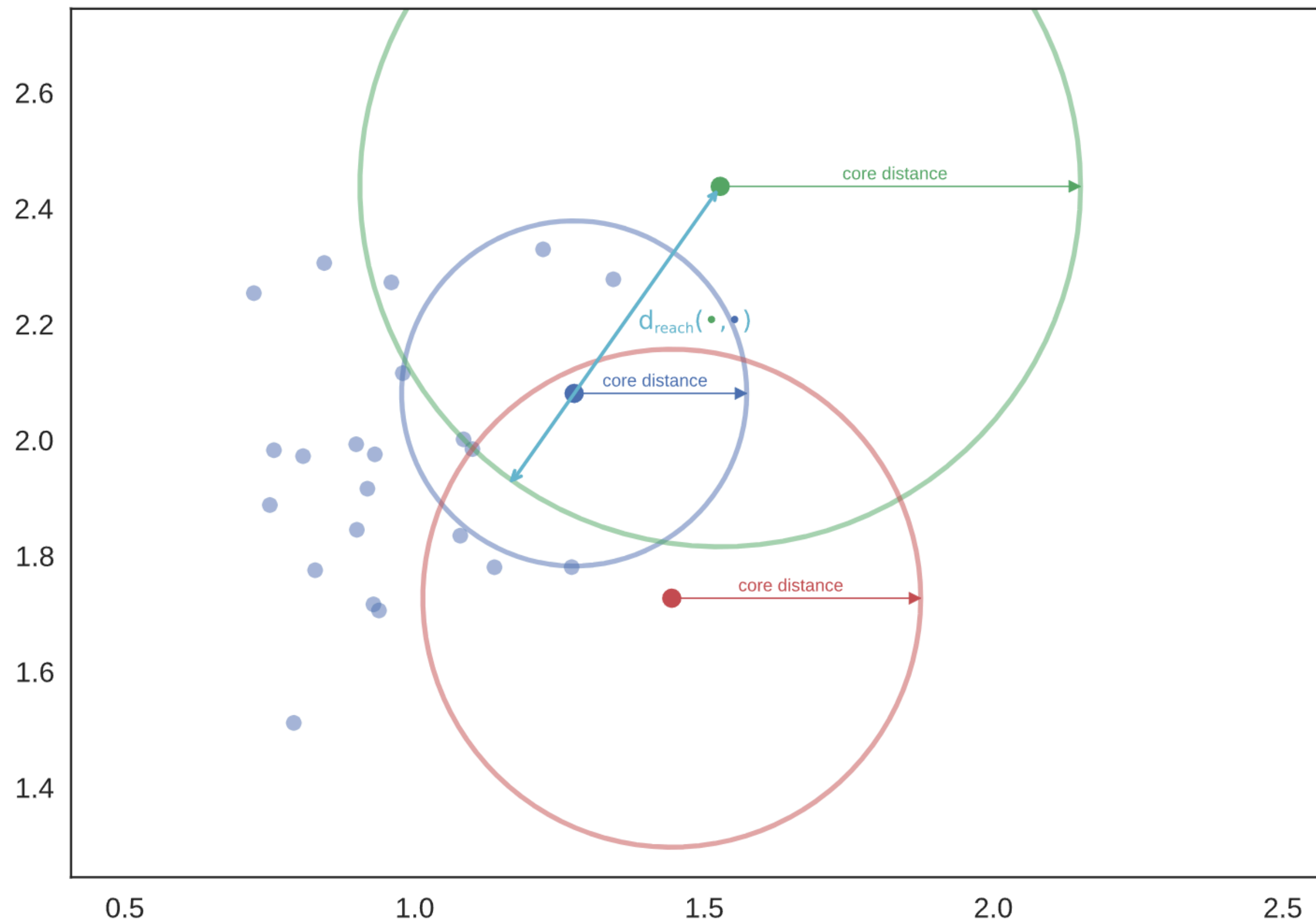
HDBSCAN



HDBSCAN



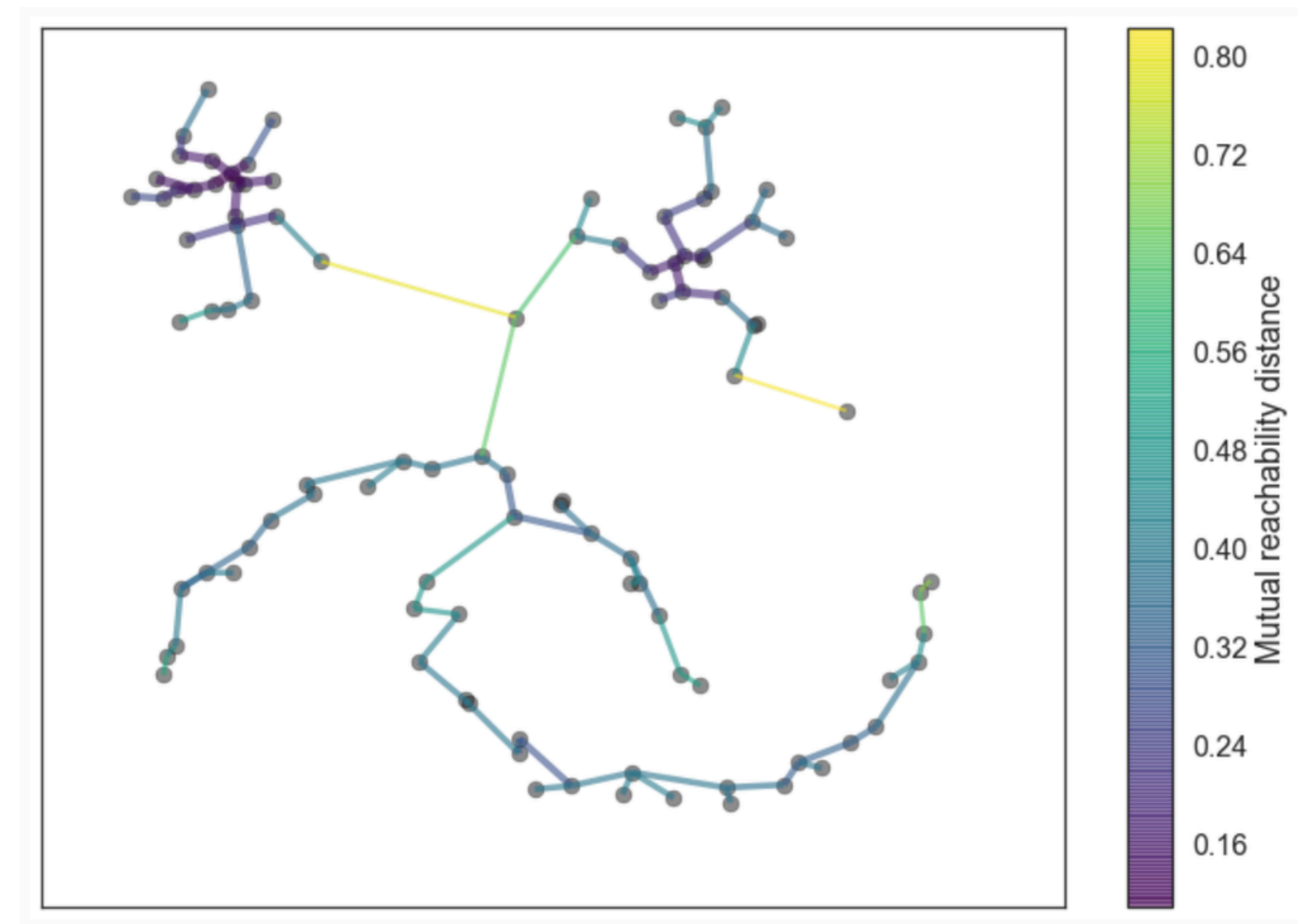
HDBSCAN



HDBSCAN

3. Construye un grafo y un árbol jerárquico

- Vamos a construir un grafo de conectividad sobre la base de la mutual reachability distance.
- Para esto usamos el **minimum spanning tree** usando el **algoritmo de Prim**. El invariante del algoritmo de Prim es agregar la arista de menor peso que conecta al árbol actual un nodo que no está conectado.

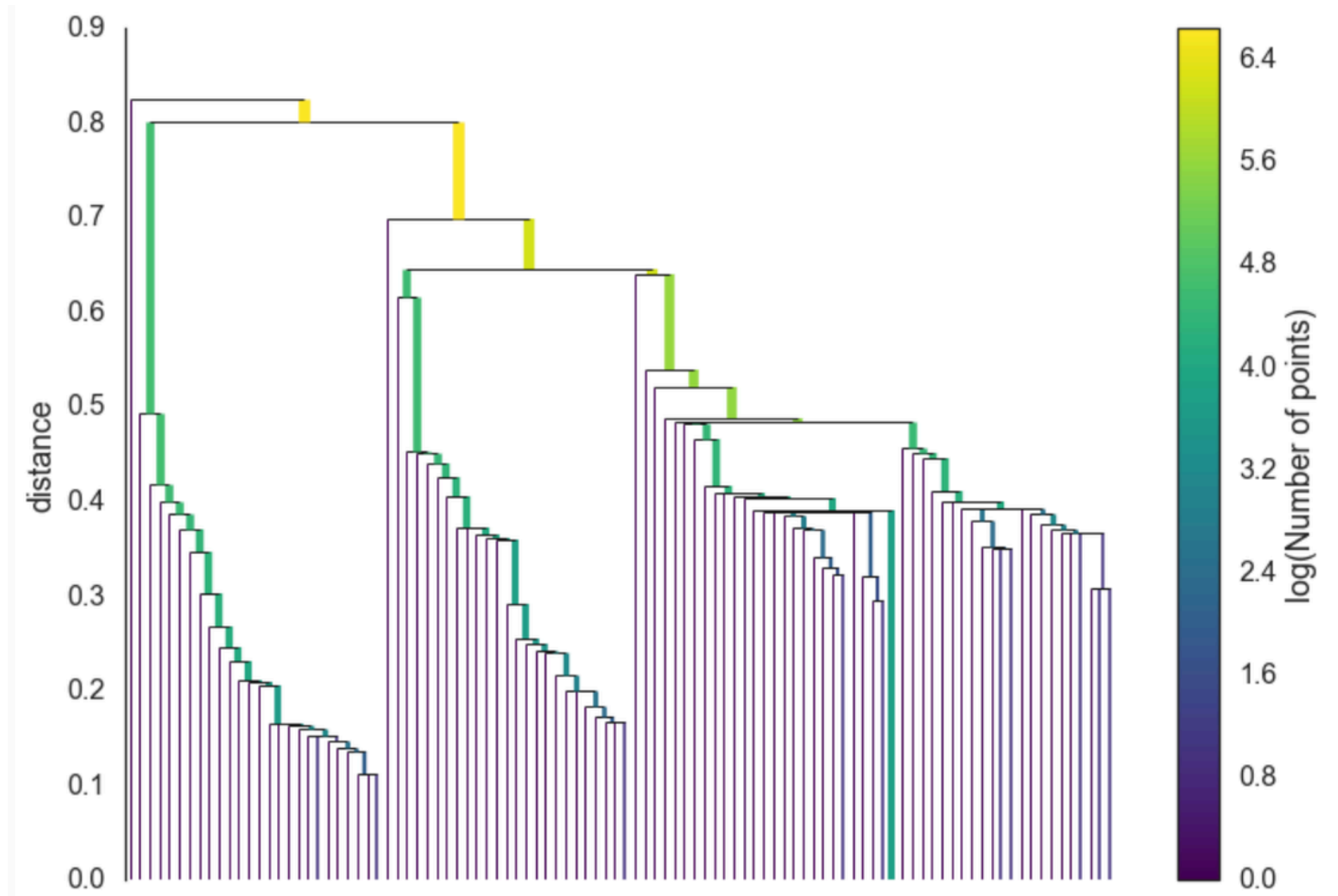


HDBSCAN

3. Construye un grafo y un árbol jerárquico

Dado el árbol de expansión mínima, el siguiente paso es **convertirlo en una jerarquía de componentes conectados**.

Esto se hace más fácilmente en orden inverso: se ordenan las aristas del árbol por distancia y luego se itera sobre ellas, **creando un nuevo clúster fusionado por cada arista**.

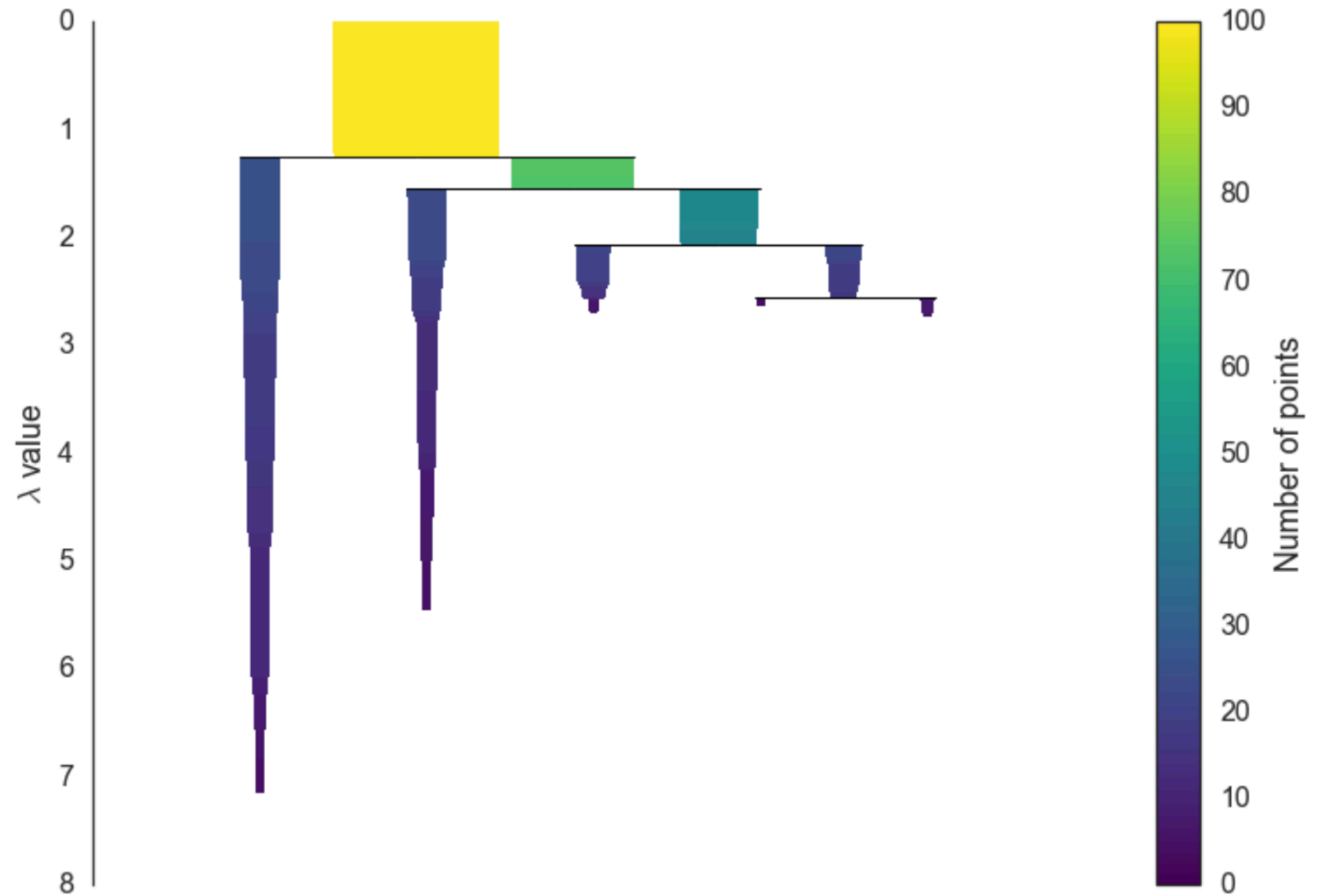


HDBSCAN

4. Condensar árbol

Condensaremos bottom-up el dendrograma con una condición de **minimum cluster size**.

- Si un cluster tiene menos datos, se mezcla con su cluster padre (merge-up).
- Si un split produce dos clusters que cumplen con la condición de tamaño, el split se mantiene



HDBSCAN

5. Extraer clusters

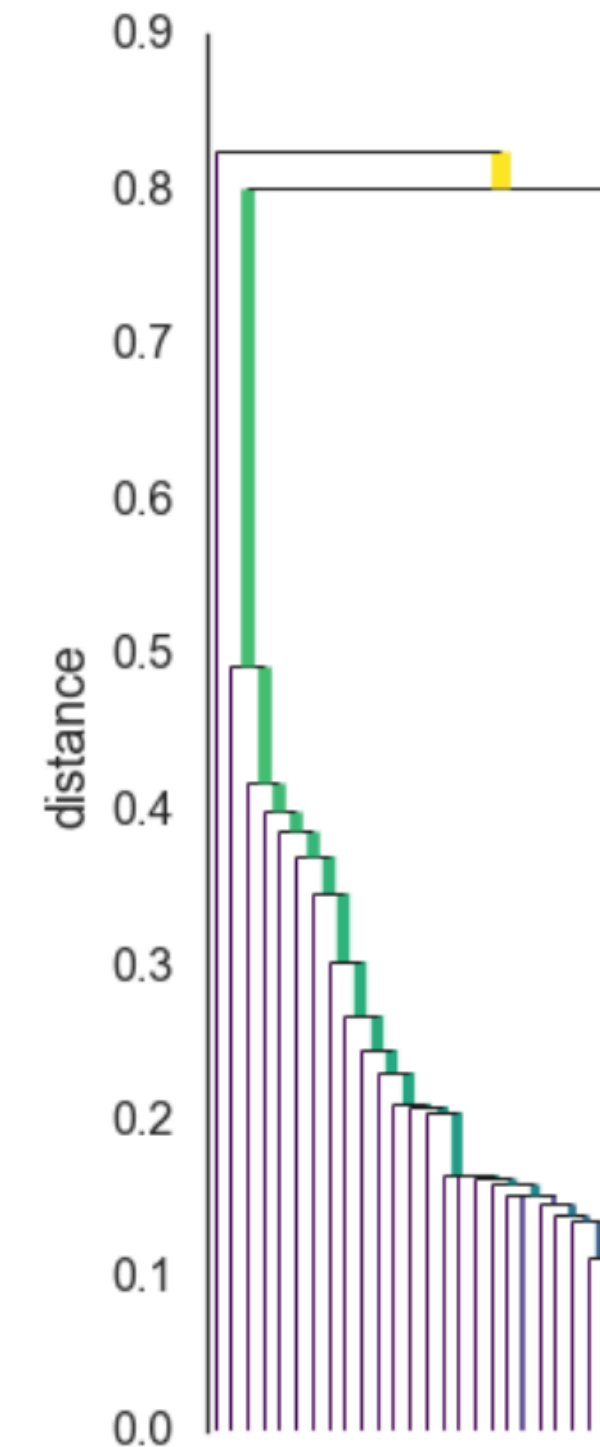
Se usa el concepto de **cluster estable** para indicar cuando un cluster sobrevive a los splits definidos por un umbral de distancia en el dendrograma. Es inverso a la distancia, por lo que se define la variable lambda como:

$$\lambda = \frac{1}{\text{distancia}}$$

Hacemos un recorrido top-down del dendrograma. Vamos a tener un lambda de nacimiento del cluster, y en la medida que d disminuya (cortamos más abajo) en algún momento el cluster va a morir.

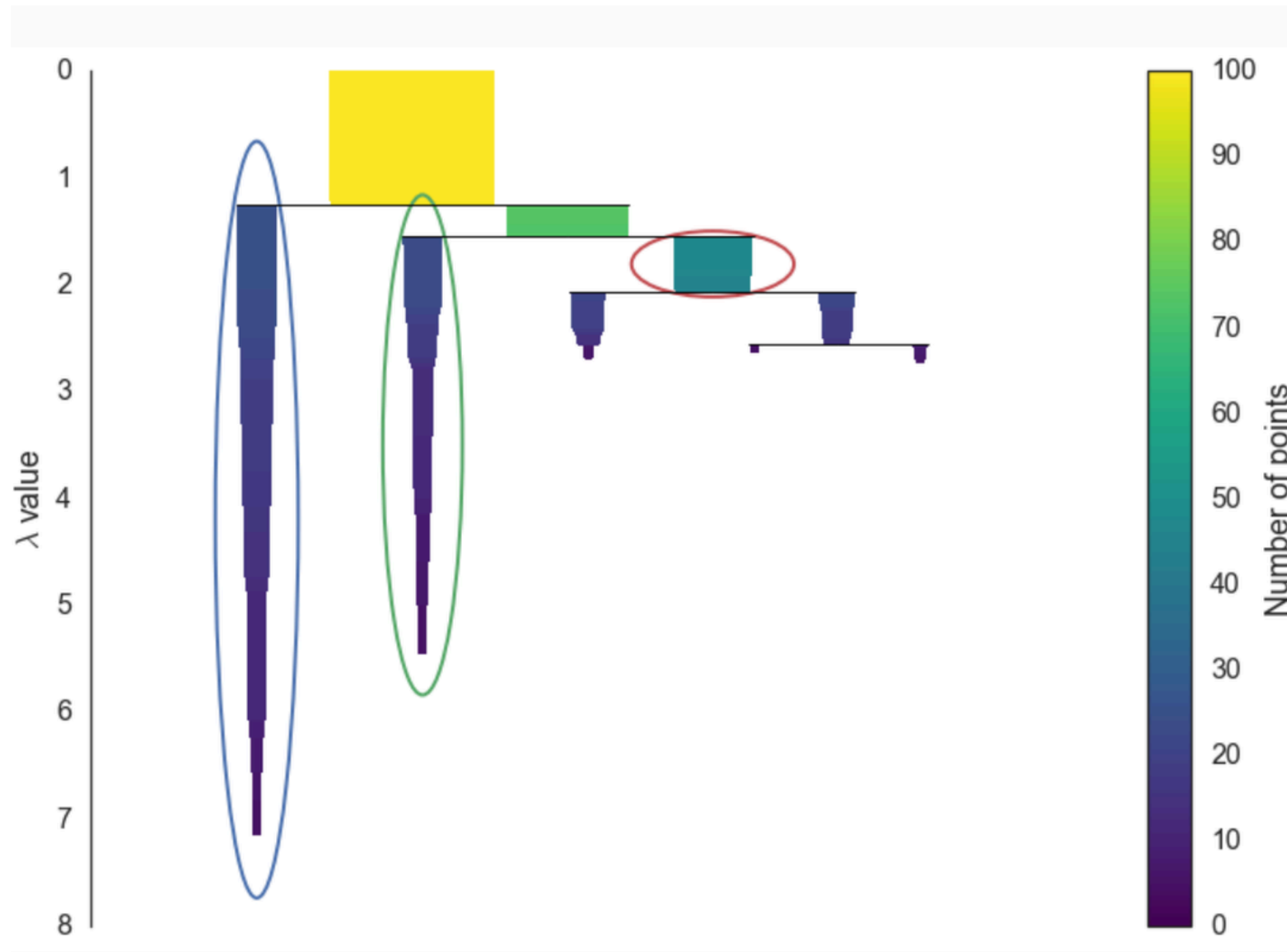
Para cada dato del cluster vamos a medir cuando sale del cluster (lambda del dato), y calcularemos la estabilidad del cluster según:

$$\text{Stability}(C) = \sum_{p \in C} (\lambda_{\text{death}}(p) - \lambda_{\text{birth}}(p))$$



HDBSCAN

5. Extraer clusters



HDBSCAN

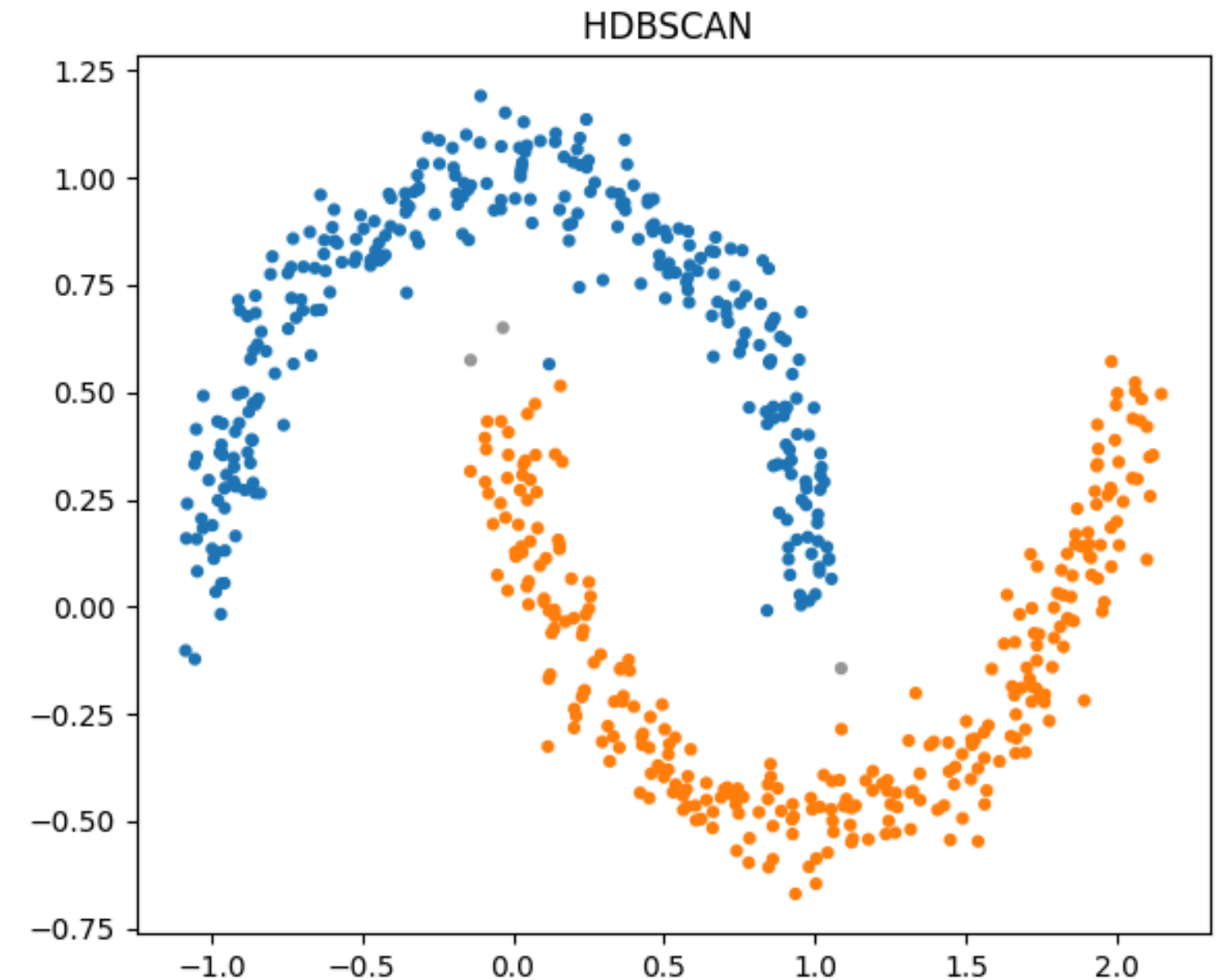
```
from sklearn.datasets import make_moons
import hdbscan

X, _ = make_moons(n_samples=600, noise=0.07, random_state=42)

# HDBSCAN
clusterer = hdbscan.HDBSCAN(min_cluster_size=20, min_samples=5)
labels = clusterer.fit_predict(X)
probs = clusterer.probabilities_

plt.scatter(X[:,0], X[:,1], s=12, c=colors)
plt.title("HDBSCAN ")
plt.tight_layout(); plt.show()

array([[1.          , 1.          , 1.          , 1.          , 1.          ,
        1.          , 1.          , 1.          , 1.          , 1.          ,
        0.76670022, 1.          , 1.          , 1.          , 1.          ,
        1.          , 1.          , 1.          , 1.          , 1.          ,
        1.          , 1.          , 1.          , 1.          , 0.97069817,
        1.          , 1.          , 1.          , 1.          , 0.74991879])
```



MINERÍA DE DATOS

Maximiliano Ojeda

muojeda@uc.cl



IIC-2433